

풀 버전 라이브 스트리밍 영상을 롱-폼 영상으로 요약해주는 서비스



저자 1 201724429 김영빈 kimyb0330@pusan.ac.kr

저자 2 202155567 손재우 jws020319@pusan.ac.kr

저자 3 202155573 엽도엽 yeom5122@pusan.ac.kr

지도교수 전상률

목 차

1. 서론.....	1
1.1. 연구 배경.....	1
1.2. 기존 문제점	2
1.3. 연구 목표.....	2
2. 연구 배경.....	4
2.1. 시스템 요구사항 및 구조 설계 과정.....	4
2.1.1. 영상 분석 방식.....	4
2.1.2. 다중 정보 활용 구간 판단.....	4
2.1.3. 문맥 고려 구간 추천	5
2.1.4. 사용자 선택 가능 편집 구조	5
2.1.5. 로컬 중심 시스템 구조.....	6
2.1.6. 외부 도구 및 음성 전사 구조.....	6
2.2. 관련 기술 조사 및 선정	6
2.2.1. UI 및 백엔드	7
2.2.2. 영상 및 데이터 수집·처리	7
2.2.3. 음성 전사	8
2.2.4. LLM 기반 내용 분석.....	8
2.2.5. DB 및 사용자 인증.....	9
2.2.6. GPU 연산 환경.....	9
2.2.7. 서버 및 배포	9
3. 연구 내용.....	11

3.1.	시스템 설계 및 구현	11
3.1.1.	전체 시스템 구조	11
3.1.2.	프로젝트 워크스페이스 및 레포지토리 구성	13
3.1.3.	영상 및 분석 데이터 수집	13
3.1.4.	음성 전사 및 스크립트 처리	14
3.1.5.	LLM 기반 내용 분석	16
3.1.6.	주요 구간 점수 산정 알고리즘	19
3.1.7.	주요 구간 추천 및 문맥 보정	23
3.1.8.	사용자 편집 및 영상 렌더링	25
3.1.9.	서버 통신 및 데이터 관리	26
3.1.10.	배포 구조	27
3.2.	시스템 동작 과정 및 사용자 인터페이스	27
3.2.1.	LLM 공급자 선택 및 영상 URL 입력	28
3.2.2.	영상 메타데이터 획득 및 추가 자료 선택	29
3.2.3.	댓글·채팅·자막 확인 및 Whisper STT 설정	30
3.2.4.	스크립트 소스 및 분석 조건 설정	32
3.2.5.	목표 길이 설정 및 챕터·점수 미리보기	33
3.2.6.	AI 자동 선택 확인 및 선택 편집	34
3.2.7.	렌더링 및 결과물 확인	35
4.	연구 결과 분석 및 평가	37
4.1.	목표 편집 길이에 따른 결과 분석	37
4.2.	구성원별 역할 및 개발 수행 내역	45
4.3.	산학협력 멘토 의견 및 반영 결과	46
4.3.1.	독자적인 모델 구축 및 모델 고도화	47

4.3.2.	하이라이트 추출 결과 및 도메인별 검증	47
4.3.3.	사용자 수동 편집 기능	48
4.3.4.	서비스 대상 및 저작권 문제	48
4.4.	종합 평가 및 한계	49
5.	결론 및 향후 연구 방향	51
5.1.	결론	51
5.2.	향후 연구 방향	52
6.	참고 문헌	53

1. 서론

1.1. 연구 배경

라이브 스트리밍(Live Streaming)은 인터넷을 통해 영상 콘텐츠를 실시간으로 송출하는 기술로, 초고속 통신망의 보급과 영상 촬영·송출 장비 및 플랫폼의 발전에 따라 누구나 비교적 손쉽게 생방송 콘텐츠를 제작할 수 있는 환경이 조성되었다. 특히 유튜브를 비롯한 온라인 동영상 플랫폼이 대중화되면서 개인 창작자와 1인 방송 종사자가 지속적으로 증가하고 있으며, 라이브 스트리밍은 주요 콘텐츠 제작 방식 중 하나로 자리 잡고 있다.

실제로 국세청의 '2020~2024년 귀속분 1인 미디어 창작자 수입 현황'에 따르면 종합소득세를 신고한 유튜버는 2020년 9,449명에서 2021~2022년 1만 명대, 2023년 2만 명대를 거쳐 2024년에는 3만 4,806명으로 증가하였다.[1] 이는 1인 미디어 콘텐츠 제작 시장이 빠른 속도로 확대되고 있음을 보여주며, 콘텐츠 생산량 증가와 함께 후속 영상 편집에 대한 수요 역시 지속적으로 증가할 가능성이 높음을 시사한다.

라이브 스트리밍 콘텐츠는 일반적으로 수십 분에서 수 시간 이상 지속되기 때문에 방송 전체가 동일한 수준의 정보성과 흥미를 유지하기 어렵다. 이에 따라 방송 종료 후 주요 구간과 핵심 내용을 선별하여 하나의 편집 영상으로 재구성하는 과정이 필요하다. 특히 원본 라이브 스트리밍을 처음부터 끝까지 확인하고 적절한 편집 구간을 선정해야 하므로 일반적인 촬영 영상보다 편집에 많은 시간이 소요될 수 있다.

이러한 영상 편집 작업을 전문 편집자에게 의뢰할 경우 콘텐츠 제작자에게 지속적인 비용 부담이 발생한다. 관련 보도에 따르면 유튜브 영상편집자의 편집 단가는 완성 영상 기준 분당 약 1만 원 수준으로 형성되어 있으며,[2] 장시간의 라이브 스트리밍을 지속적으로 편집하는 경우 개인 창작자에게 상당한 비용 부담으로 작용할 수 있다.

개인 창작자가 편집 비용을 줄이기 위해 직접 편집하는 방법도 있으나, 이 경우 영상 편집 기술을 별도로 습득해야 하고 방송 외적인 작업에 상당한 시간을 투입해야 한다. 특히 지속적으로 콘텐츠를 제작하는 1인 방송 종사자에게는 방송 기획, 촬영, 시청자 관리와 함께 편집 업무까지 직접 수행해야 한다는 점이 부담으로 작용할 수 있다.

따라서 장시간의 라이브 스트리밍으로부터 다양한 정보를 자동으로 분석하고 핵심 구간을 선별하여 편집할 수 있는 기술에 대한 필요성이 증가하고 있다. 특히 이러한 기술을 낮은 비용으로 활용할 수 있다면 전문 편집 인력을 지속적으로 고용하기 어려운 개인 창작자와 소규모 콘텐츠 제작자의 영상 편집 부담을 완화할 수 있을 것으로 기대된다.

1.2. 기존 문제점

현재 장시간 영상의 편집을 자동화하기 위한 대표적인 AI 기반 서비스로는 OpusClip, Vizard, Descript 등이 있다. 이들 서비스는 긴 영상에서 주요 구간을 탐색하고 이를 짧은 클립으로 재구성하는 기능을 제공한다.

OpusClip은 긴 영상에서 주요 구간을 분석하여 YouTube Shorts, TikTok 등에서 활용할 수 있는 숏폼 영상을 자동 생성하는 서비스이며,[3] Vizard 역시 장시간 영상을 여러 개의 짧은 클립으로 변환하는 기능을 중심으로 제공한다.[4] Descript 또한 AI Clips 기능을 통해 긴 영상이나 팟캐스트에서 주요 구간을 추출하고 짧은 영상으로 제작할 수 있도록 지원한다.[5]

이러한 기존 솔루션의 공개 기능 설명에서는 장시간 원본에서 개별적인 숏폼 영상이나 하이라이트 클립을 생성하는 활용이 두드러진다. 반면 본 연구는 장시간 영상의 여러 핵심 구간을 문맥상 필요한 연결 구간과 함께 추천하고, 사용자가 이를 검토·확정하여 하나의 목표 길이 편집본으로 재구성하는 과정에 초점을 둔다.

특히 기존 방식을 활용하여 하나의 롱폼 편집본을 제작하려는 경우, 자동으로 생성된 여러 클립을 사용자가 다시 검토하고 순서를 조정하는 등의 추가 편집 과정이 필요할 수 있다. 따라서 장시간의 라이브 스트리밍을 대상으로 전체 내용의 흐름을 고려하여 주요 구간을 추천하고, 사용자의 선택을 반영하여 이를 하나의 롱폼 영상으로 재구성할 수 있는 별도의 접근이 필요하다.

1.3. 연구 목표

본 연구의 목표는 장시간의 라이브 스트리밍 다시보기 영상(VOD)에서 다양한 정보를 분석하여 주요 구간을 자동으로 추천하고, 사용자가 이를 검토·확정하여 설정한 목표 길이를 기준으로 롱폼(Long-form) 영상으로 재구성할 수 있는 영상 요약·편집 서비스를 개발하는 것이다.

본 연구는 종료된 라이브 스트리밍 다시보기 영상(VOD)을 주된 대상으로 하며, 구현된 시스템은 동일한 입력 조건을 만족하는 일반 업로드 YouTube 영상에도 적용할 수 있도록 구성하였다. 다만 진행 중인 실시간 방송은 지원 대상에 포함하지 않는다.

기존의 자동 영상 편집 서비스가 주로 짧은 하이라이트나 숏폼 콘텐츠 생성에 초점을 두고 있는 것과 달리, 본 연구에서는 라이브 스트리밍 전체의 내용과 흐름을 고려하여

시청자가 원본 영상을 직접 시청하지 않고도 주요 내용을 파악할 수 있는 수준의 편집본을 생성하는 것을 목표로 한다.

이를 위해 주로 자동 생성 캡션 또는 STT(Speech-to-Text)를 통해 확보한 스크립트를 기반으로 LLM이 영상의 챕터와 세부 구간을 구성하고 각각의 내용적 중요도를 평가한다. 또한 이용 가능한 경우 채팅 활성화 수준과 댓글 타임스탬프 등의 사용자 반응 정보, 음량 변화와 같은 음향 정보, 시청 히트맵과 같은 시청 행동 정보를 함께 활용하여 주요 구간을 종합적으로 평가한다. 이후 시스템이 추천한 구간을 사용자가 검토·확정하면, 선택된 구간을 원본 영상의 시간 순서에 따라 결합하고 자막을 합성하여 하나의 롬폼 요약 영상을 생성하도록 한다.

또한 본 연구에서는 고가의 영상 편집 소프트웨어나 전문 편집 인력에 대한 의존도를 줄이고, 개인 방송 종사자와 소규모 콘텐츠 제작자도 최소한의 비용으로 이용할 수 있는 서비스 환경을 구축하는 것을 목표로 한다. 이를 통해 라이브 스트리밍 종료 후 반복적으로 발생하는 영상 탐색, 주요 구간 선정 및 렌더링 과정의 상당 부분을 자동화하고, 콘텐츠 제작에 필요한 시간과 비용을 절감하고자 한다.

궁극적으로는 콘텐츠 제작자가 영상 편집 과정에 투입하는 자원을 최소화하고 방송 기획 및 콘텐츠 제작 자체에 더욱 집중할 수 있도록 지원하는 것을 본 연구의 최종 목표로 한다.

2. 연구 배경

2.1. 시스템 요구사항 및 구조 설계 과정

본 연구에서는 장시간의 라이브 스트리밍 영상을 효율적으로 요약·편집하기 위해 시스템 구축에 앞서 처리 시간, 연산 비용, 주요 구간 판단 방식, 사용자 개입 범위, 영상 데이터 관리 방식 등을 주요 제약사항으로 검토하였다. 초기 착수 단계에서 검토한 여러 방안은 실제 구현 과정에서 시스템의 비용과 운영 환경을 고려하여 수정·구체화하였으며, 최종적으로는 사용자 PC에서 영상 처리를 수행하고 외부 서버와 AI 서비스를 필요한 범위에서 활용하는 구조를 채택하였다.

2.1.1. 영상 분석 방식

장시간의 고화질 영상 데이터를 직접 AI 모델에 입력하여 분석하는 방식은 많은 연산 자원과 긴 처리 시간을 필요로 한다. 또한 영상의 시각적 특징을 기반으로 주요 구간을 판단하는 전용 AI 모델을 직접 구축할 경우 학습 데이터 확보, 모델 개발, 학습 및 배포에 추가적인 비용과 자원이 요구된다.

이에 본 연구에서는 영상 자체의 모든 시각 정보를 직접 분석하기보다, 영상 및 플랫폼에서 확보할 수 있는 텍스트·사용자 반응·음향·시청 행동 정보를 중심으로 주요 구간을 판단하도록 방향을 설정하였다. 자동 생성 캡션 또는 음성 전사를 통해 확보한 스크립트를 기본적인 내용 분석 자료로 활용하고, LLM을 통해 영상의 내용적 구조와 각 구간의 중요도를 평가하도록 하였다. 실제 시스템은 스크립트를 챕터와 섹션 단위로 구성하여 중요도를 평가하고, 이를 주요 구간 추천에 활용한다.

2.1.2. 다중 정보 활용 구간 판단

하나의 분석 정보만으로 주요 구간을 판단할 경우 영상에서 발생하는 다양한 형태의 중요 장면을 충분히 반영하기 어렵다고 판단하였다. 예를 들어 스크립트의 내용만으로는 시청자의 반응이 크게 나타난 구간을 놓칠 수 있으며, 반대로 시청자 반응만으로 판단할 경우 내용적으로 중요한 부분이 제외될 가능성이 있다.

따라서 본 연구에서는 판단 근거를 확대하여 다양한 유형의 주요 장면을 반영하기 위해 여러 종류의 정보를 함께 활용하도록 설계하였다. LLM이 평가한 챕터와 섹션의 내용

적 중요도뿐만 아니라 댓글 타임스탬프와 채팅 활성화 수준 등의 사용자 반응 정보, 음량 변화와 같은 음향 정보, 시청 히트맵과 같은 시청 행동 정보를 종합적으로 활용한다. 실제 시스템에서는 이용 가능한 정보만을 총점에 반영하며, 히트맵, 댓글, LLM 중요도, 채팅, 음량 등에 서로 다른 가중치를 적용하여 구간별 점수를 계산한다.

2.1.3. 문맥 고려 구간 추천

주요 구간의 점수만을 기준으로 개별 구간을 추출하여 단순히 연결할 경우, 앞뒤 내용이 생략되어 결과 영상의 흐름이 부자연스러워질 수 있다. 특히 장시간의 방송에서는 특정 장면 자체보다 그 장면에 도달하기까지의 대화나 사건 전개가 내용 이해에 필요한 경우가 있다.

이를 보완하기 위해 영상의 내용을 큰 주제 단위인 챕터와 세부 의미 단위인 섹션으로 구성하고, 높은 점수를 받은 섹션만을 독립적으로 선택하는 대신 내용을 이해하기 위해 필요한 연결 구간을 함께 고려하도록 설계하였다. 실제 구현에서는 점수가 높은 섹션을 기준으로 같은 챕터 내에서 이해에 필요한 연결 섹션을 LLM에 질의하여 추천 결과에 포함할 수 있도록 구성하였다.

2.1.4. 사용자 선택 가능 편집 구조

AI가 추천하는 편집 구간이 모든 사용자의 의도와 항상 일치한다고 보장할 수는 없다. 특히 사용자가 원하는 장면이나 방송의 핵심이라고 판단하는 부분은 개인의 편집 목적에 따라 달라질 수 있으며, AI가 모든 편집 결과를 일방적으로 확정할 경우 결과물에 대한 사용자 만족도가 낮아질 수 있다.

따라서 본 연구에서는 AI가 최종 영상을 완전히 자동으로 확정하는 방식보다, 주요 구간을 우선 추천한 뒤 사용자가 이를 검토하고 최종 선택할 수 있는 구조를 채택하였다. 사용자는 추천된 구간과 목표 영상 길이를 조정할 수 있으며, 최종 선택이 완료된 이후 시스템이 해당 구간을 결합하여 결과 영상을 생성한다. 현재 구현에서도 추천 결과는 자동으로 확정되는 최종 영상이 아니라 사용자가 수정할 수 있는 초안의 역할을 한다.

편집 영상은 일반적으로 지나치게 길지 않으면서도 주요 내용을 충분히 전달할 수 있다고 판단한 10~30분 내외를 대표적인 활용 범위로 설정하였다. 다만 이를 고정된 길이로 제한하지 않았으며, 현재 시스템에서는 사용자가 원본 영상의 길이를 초과하지 않는

범위에서 1분부터 최대 120분까지 목표 길이를 설정할 수 있도록 하였다.

2.1.5. 로컬 중심 시스템 구조

장시간 영상 파일은 데이터의 크기가 크기 때문에 원본 영상을 중앙 서버로 전송하고 저장하는 방식은 대규모 저장 공간과 네트워크 자원을 요구한다. 또한 사용자가 업로드하거나 시청한 원본 영상을 중앙 서버에서 보관할 경우 개인정보 및 프라이버시와 관련된 부담이 발생할 수 있다. 이와 함께 데이터센터의 IP 주소를 통한 YouTube 접근이 제한되는 경우가 있어, 중앙 서버에서 모든 영상 다운로드를 수행하는 방식은 서비스 안정성 측면에서도 제약이 있다고 판단하였다.

이에 최종 시스템에서는 원본 영상 다운로드, 분석을 위한 보조 자료 생성, 편집 및 렌더링을 사용자 PC에서 수행하도록 구성하였다. 중앙 서버는 사용자 인증, 외부 LLM 및 원격 STT 서비스의 중계, 완료된 작업 이력 관리 등을 담당하고, 원본 영상과 완성된 결과 영상은 중앙 서버의 영구 저장 대상으로 사용하지 않는다.

2.1.6. 외부 도구 및 음성 전사 구조

현재 시스템은 yt-dlp를 Python 라이브러리에 직접 포함하지 않고 별도의 실행 파일 형태로 사용하며, 사용자가 별도의 업데이터를 통해 yt-dlp 및 FFmpeg 관련 도구를 갱신할 수 있도록 구성하였다. 이를 통해 프로그램 전체를 다시 설치하지 않고도 비교적 자주 변경되는 외부 도구를 최신 상태로 유지할 수 있도록 하였다.

음성 전사 방식은 비용을 고려하여 RunPod 환경에서 WhisperX의 large-v3 모델을 이용하며, 단어 단위의 시간 정렬 결과를 영상 편집에 활용할 수 있도록 구성되어 있다.

이와 같은 설계 과정을 통해 본 연구에서는 대용량 영상 자체를 중앙에서 분석·저장하는 구조보다 사용자 PC의 로컬 처리 능력을 활용하고, 스크립트와 다양한 반응 정보를 결합하여 주요 구간을 추천하는 구조를 최종적으로 채택하였다. 또한 AI의 판단을 사용자가 수정할 수 있도록 함으로써 자동화에 따른 작업 효율을 확보하면서도 사용자의 편집 의도를 반영할 수 있도록 시스템 구조를 구체화하였다.

2.2. 관련 기술 조사 및 선정

본 연구에서는 장시간 영상의 수집, 음성 전사, 내용 분석, 사용자 인터페이스, 영상 편

집 및 배포 환경을 구성하기 위해 다양한 기술을 사전에 조사하였다. 초기 단계에서는 기능 구현 가능성을 중심으로 여러 기술을 폭넓게 검토하였으며, 실제 구현 과정에서 각 기술의 적합성, 비용, 정확도, 실행 환경 및 유지보수 편의성을 고려하여 일부 기술은 유지하고 일부는 다른 기술로 교체하였다. 각 기술은 단순히 사용 여부만을 결정한 것이 아니라, 본 연구에서 요구하는 기능을 수행하기에 적합한지를 비교하여 선정하였다.

2.2.1. UI 및 백엔드

React는 웹 화면을 여러 개의 독립적인 구성 요소로 나누어 개발할 수 있도록 지원하는 사용자 인터페이스 개발 기술이다. 본 연구에서는 사용자가 영상 URL을 입력하고, 분석 설정을 변경하며, 추천 구간을 확인하고 최종 편집 구간을 선택할 수 있는 웹 기반 화면을 구성하기 위해 React를 검토하고 적용하였다.

Python은 다양한 영상·음성 처리 및 AI 관련 라이브러리를 활용할 수 있는 범용 프로그래밍 언어이며, FastAPI는 Python으로 웹 API 서버를 구현하기 위한 프레임워크이다. 본 연구에서는 사용자 PC에서 실행되는 로컬 처리 엔진과 중앙 서버의 API를 구성하기 위해 Python과 FastAPI를 사용하였다. 이를 통해 브라우저 인터페이스와 로컬 영상 처리 기능, 중앙 서버 간 요청을 연결할 수 있도록 하였다.

2.2.2. 영상 및 데이터 수집·처리

본 연구에서는 영상 자체뿐만 아니라 자막, 자동 생성 캡션, 채팅 리플레이, 히트맵 등 주요 구간 분석에 활용할 수 있는 다양한 자료가 필요하였다. YouTube Data API만으로는 이러한 정보를 충분히 확보하기 어렵다고 판단하여, 보다 폭넓은 영상 및 부가 자료를 수집할 수 있는 yt-dlp로 중심 기술을 선택하였다.

yt-dlp는 YouTube를 비롯한 온라인 동영상 플랫폼에서 영상 파일과 자막, 각종 메타데이터를 수집할 수 있는 도구이다. 본 연구에서는 이를 통해 원본 영상과 분석에 필요한 부가 자료를 확보하도록 하였다.

영상 처리 기술로는 FFmpeg를 선택하였다. 영상과 음성의 변환, 인코딩, 구간 결합, 자막 합성 등을 수행할 수 있는 미디어 처리 도구이며, FFprobe는 영상과 음성 파일의 길이, 코덱, 스트림 정보 등을 확인하는 도구이다. 이에 최종적으로는 FFmpeg와 FFprobe를 중심으로 영상 처리 구조를 구성하였다.

2.2.3. 음성 전사

STT(Speech-to-Text)는 음성 데이터를 텍스트로 변환하는 기술로, 본 연구에서는 영상의 발화 내용을 스크립트 형태로 확보하기 위해 필요하다. 또한 실제 편집에 활용하기 위해서는 단순한 텍스트뿐 아니라 각 문장과 단어가 원본 영상의 어느 시점에 위치하는지에 대한 정확한 타임스탬프가 중요하다.

음성 인식 기술로는 WhisperX를 선택하였다. WhisperX는 Whisper 기반 전사 결과에 별도의 강제 정렬 과정을 적용하여 단어 단위 타임스탬프를 보다 정밀하게 확보할 수 있도록 하는 기술이다. 현재 시스템에서는 세그먼트 및 단어 단위 시간 정보를 생성하고, 이를 실제 영상 편집 구간과 연결하는 데 활용한다.

2.2.4. LLM 기반 내용 분석

대규모 언어 모델(LLM)은 대량의 텍스트를 기반으로 문맥을 이해하고 요약, 분류, 판단 등의 자연어 처리 작업을 수행할 수 있는 AI 모델이다. 본 연구에서는 장시간 영상에서 확보한 스크립트의 주제를 구분하고, 각 구간의 내용적 중요도를 평가하며, 주요 구간의 내용을 이해하기 위해 필요한 연결 구간을 판단하는 데 LLM을 활용하였다.

비용 효율성과 긴 컨텍스트 처리 능력을 고려하여 Gemini 3.5 Flash-Lite와 DeepSeek V4 Flash를 활용하도록 구성하였다. Gemini 3.5 Flash-Lite는 대량의 요청을 낮은 비용으로 처리하는 용도를 목표로 제공되는 모델이며, API 가격은 100만 입력 토큰당 0.30\$, 100만 출력 토큰당 2.50\$이다. 이러한 가격 구조는 장시간 스크립트와 같이 입력 데이터의 양이 큰 작업에 활용하기 적합하다고 판단하였다.

DeepSeek V4 Flash는 빠른 처리와 비용 효율성을 중점으로 제공되는 대규모 언어 모델로, 최대 100만 토큰의 컨텍스트 길이를 지원한다. 또한 하나의 모델에서 일반 응답과 추론 모드를 모두 지원하며, 본 연구에서 요구하는 장시간 스크립트 분석에 충분한 입력 범위를 제공한다. DeepSeek API는 시간대에 따라 피크 및 비피크 요금을 구분하며, V4 Flash의 입력 비용은 100만 토큰당 비피크 시간 0.22\$, 피크 시간 0.44\$이고, 출력 비용은 각각 0.66\$와 1.32\$이다.

두 모델은 스크립트를 큰 주제 단위의 챕터와 세부 의미 단위의 섹션으로 구분하고, 각 챕터와 섹션의 내용적 중요도를 평가하는 데 활용된다. 또한 점수가 높은 주요 구간이 선정된 이후에는 해당 구간의 내용 이해에 필요한 연결 섹션을 판단하는 데에도 LLM

을 활용한다.

2.2.5. DB 및 사용자 인증

데이터베이스는 PostgreSQL 기반 데이터베이스를 제공하는 Supabase를 선택했다. 사용자 인증 기능을 함께 제공하므로 별도의 인증 시스템을 구축하는 부담을 줄일 수 있었다. 이에 중앙 서버에서는 Supabase를 이용하여 사용자 인증과 완료 작업 이력을 관리하도록 구성하였다.

사용자 PC에서는 SQLite를 활용한다. SQLite는 별도의 데이터베이스 서버 없이 하나의 파일 형태로 데이터를 저장할 수 있는 경량 관계형 데이터베이스로, 로컬 완료 작업 정보를 관리하는 데 적합하다고 판단하였다.

2.2.6. GPU 연산 환경

GPU는 대량의 병렬 연산에 적합한 하드웨어로, Whisper와 같은 대형 음성 인식 모델을 빠르게 실행하는 데 활용된다. 그러나 본 연구의 개발 환경에는 해당 모델을 원활하게 실행할 수 있는 충분한 GPU 자원이 없었기 때문에 외부 GPU 환경을 활용하는 방안을 검토하였다.

본 연구에서는 RunPod의 서버리스 GPU 환경으로 변경하였다. 서버리스 방식은 전사 요청이 발생하는 시점에만 GPU 자원을 사용할 수 있어 개발 환경에 고성능 GPU가 없어도 원격으로 WhisperX를 실행할 수 있으며, GPU 서버를 상시 운영하는 방식보다 비용 부담도 줄일 수 있다.

2.2.7. 서버 및 배포

Docker는 프로그램과 실행에 필요한 환경을 하나의 컨테이너 단위로 구성하여 서로 다른 서버에서도 동일한 실행 환경을 유지할 수 있도록 하는 기술이다. 본 연구에서는 중앙 서버와 원격 서비스의 실행 환경을 일관되게 구성하기 위한 기술로 Docker를 검토하였다.

초기 설계에서는 Nginx를 이용한 로드 밸런싱 및 리버스 프록시 구조와 여러 클라우드 서버의 활용을 고려하였다. Nginx는 외부 요청을 내부 서버로 전달하거나 여러 서버

에 분산할 수 있는 웹 서버 및 프록시 소프트웨어이다. 그러나 이후 원본 영상과 결과 영상을 중앙 서버에 저장하지 않는 로컬 중심 구조로 변경되면서 대규모 서버 구성의 필요성이 감소하였다.

현재 중앙 서버에서는 Caddy를 사용한다. Caddy는 HTTPS 연결과 리버스 프록시 기능을 제공하는 웹 서버로, 외부 요청을 FastAPI 서버로 전달하는 역할을 수행한다. 또한 원격 Whisper 전사를 위해 중앙 서버에 임시로 업로드된 음성 파일을 제한된 시간 동안 외부에서 접근할 수 있도록 제공하며, RunPod의 전사 Worker가 해당 파일을 받아 처리할 수 있도록 하는 역할도 담당한다.

Windows 클라이언트 배포 방식으로는 초기에는 pynsist를 검토하였다. pynsist는 Python 프로그램과 Python 인터프리터를 함께 묶어 Windows 설치 프로그램을 생성하는 도구이다. 그러나 사용자 PC에 이미 Python이 설치되어 있는 경우 별도의 인터프리터가 다시 설치되어 실행 환경이 중복될 가능성이 있다고 판단하였다.

이에 최종적으로 PyInstaller를 채택하였다. PyInstaller는 Python 프로그램과 필요한 라이브러리를 실행 가능한 형태로 묶어 사용자가 별도의 Python 환경을 설치하지 않고도 프로그램을 실행할 수 있도록 하는 도구이다. 현재 시스템에서는 이를 이용하여 Windows용 포터블 실행 환경을 구성하고 있다.

이와 같이 본 연구에서는 각 기술의 기본 역할을 조사한 뒤, 실제 구현 과정에서 시스템 요구사항에 적합한지를 검토하여 최종 기술을 선정하였다. 특히 영상 편집에 필요한 타임스탬프의 정밀도, 데이터 확보 범위, 개발 환경의 GPU 제약, 외부 API 이용 비용, 영상 처리 방식의 적합성, 데이터베이스 호환성 및 사용자 환경에서의 배포 편의성을 주요 판단 기준으로 활용하였다.

3. 연구 내용

3.1. 시스템 설계 및 구현

3.1.1. 전체 시스템 구조

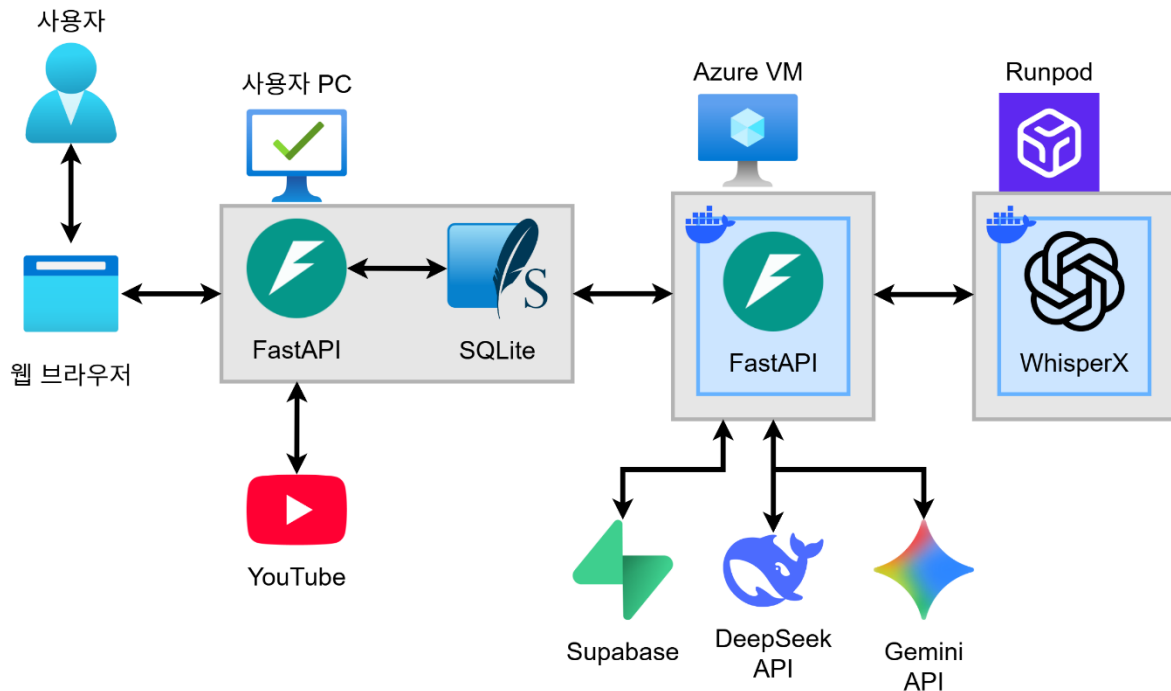


그림 1 시스템 전체 구성도

본 연구에서 개발한 영상 요약·편집 시스템은 사용자 PC에서 대부분의 영상 처리를 수행하고, 인증과 외부 AI 서비스 연동이 필요한 기능만 중앙 서버를 통해 처리하는 로컬 중심 구조로 구성하였다. 그림 1과 같이 전체 시스템은 사용자 및 웹 브라우저, 사용자 PC의 로컬 처리 환경, Azure VM의 중앙 서버, RunPod의 원격 음성 전사 환경, 그리고 YouTube, Supabase, DeepSeek API, Gemini API 등의 외부 서비스로 구성된다.

사용자는 웹 브라우저를 통해 시스템을 이용한다. 웹 브라우저는 사용자 PC에서 실행되는 FastAPI 기반 로컬 서버와 통신하며, 영상 URL 입력, 분석 설정, 진행 상태 확인, 추천 구간 검토, 최종 편집 구간 선택 등의 기능을 제공한다. 사용자 인터페이스와 로컬 서버 간 통신에는 REST API와 SSE(Server-Sent Events)를 사용하며, 실제 영상 처리 작업은 사용자 PC에서 수행된다.

사용자 PC의 로컬 처리 환경은 시스템의 주요 실행 영역이다. 이 영역에서는 YouTube 영상 및 관련 자료 수집, 스크립트 처리, 주요 구간 점수 계산, 추천 결과 생성, 사용자

선택 결과 반영 및 최종 영상 렌더링을 수행한다. 또한 작업과 관련된 일부 로컬 정보는 SQLite를 통해 관리한다. 원본 영상과 최종 편집 영상은 사용자 PC에만 저장된다.

YouTube와의 통신도 사용자 PC에서 직접 수행된다. 로컬 시스템은 원본 영상뿐 아니라 이용 가능한 경우 자막, 자동 생성 캡션, 댓글, 라이브 채팅 리플레이, 히트맵, 챕터 등의 부가 자료를 수집한다. 이와 같이 대용량 영상 데이터를 중앙 서버를 거치지 않고 사용자 PC에서 직접 수집하도록 하여 중앙 서버의 저장 공간 및 네트워크 사용량을 줄이도록 구성하였다.

사용자 PC에서 외부 AI 기능이 필요한 경우 Azure VM에서 실행되는 중앙 FastAPI 서버와 통신한다. 중앙 서버는 사용자 인증, 외부 LLM 요청 중계, 원격 음성 전사 요청 관리 및 완료 작업 이력 저장 등의 역할을 수행한다. 외부 API 인증 정보와 서버 측 설정은 중앙 서버에서 관리하며, 사용자 PC가 Gemini API나 DeepSeek API를 직접 호출하지 않도록 구성하였다.

사용자 인증 및 일부 완료 작업 이력 관리는 Supabase를 이용한다. 중앙 서버는 Supabase를 통해 사용자 인증 정보를 확인하고, 작업이 완료된 경우 일부 결과 정보와 작업 이력을 저장한다.

영상의 내용 분석에는 Gemini API와 DeepSeek API를 사용한다. 사용자 PC에서 확보한 스크립트에 대한 분석 요청은 Azure VM의 중앙 서버를 통해 선택된 LLM으로 전달된다. LLM은 영상의 내용을 챕터와 세부 섹션 단위로 구성하고 각 구간의 내용적 중요도를 평가하며, 이후 주요 구간을 추천하는 과정에서는 같은 챕터 내에서 내용 이해에 필요한 연결 구간을 판단하는 데에도 활용된다.

영상에서 사용할 자막이나 자동 생성 캡션을 이용하지 않는 경우에는 RunPod 환경의 WhisperX를 통해 원격 음성 전사를 수행한다. 사용자 PC에서 전사용 음성 파일을 생성한 뒤 중앙 서버에 임시 업로드하면, Azure VM은 해당 파일을 제한된 시간 동안 HTTPS로 접근할 수 있도록 제공한다. 이후 중앙 서버가 RunPod에 전사 작업을 요청하고, RunPod의 WhisperX는 해당 음성 파일을 받아 음성 전사와 CTC 기반 강제 정렬을 수행한다. 생성된 문장 및 단어 단위의 시간 정보는 중앙 서버를 거쳐 사용자 PC로 반환되며, 전사 작업이 완료된 뒤 중앙 서버의 임시 음성 파일은 정리된다.

전체 처리 과정은 사용자의 YouTube URL 입력에서 시작한다. 사용자 PC는 영상과 분석에 필요한 자료를 수집하고, 자막 또는 WhisperX를 통해 스크립트를 확보한다. 이후 중앙 서버를 거쳐 Gemini 또는 DeepSeek를 이용한 내용 분석을 수행하고, 로컬에서

LLM 중요도와 닷글, 채팅, 음량, 히트맵 등의 정보를 종합하여 각 섹션의 최종 점수를 계산한다. 시스템은 이 점수를 기준으로 주요 구간과 문맥상 필요한 연결 구간을 추천하며, 사용자가 최종 편집 구간을 확정하면 사용자 PC에서 FFmpeg를 이용하여 결과 영상을 생성한다.

이와 같이 본 시스템은 대용량 영상의 다운로드, 분석 보조 처리 및 렌더링은 사용자 PC에서 수행하고, 사용자 인증, LLM 분석 및 GPU 기반 음성 전사와 같이 외부 자원이 필요한 기능만 Azure VM과 외부 서비스로 분리하였다. 이를 통해 대용량 원본 영상의 중앙 서버 전송과 영구 저장을 최소화하면서도 외부 AI 기능을 활용할 수 있도록 전체 시스템을 구성하였다.

3.1.2. 프로젝트 워크스페이스 및 레포지토리 구성

본 연구의 개발 워크스페이스는 ave-workspace를 기준으로 구성하였다. 워크스페이스 루트의 setup.bat을 실행하면 modules 디렉터리 아래에 ave-client, ave-server, ave-whisper-api, ave-dist의 네 개 독립 Git 저장소를 배치할 수 있도록 하였다. 다만 해당 스크립트는 소스 저장소를 구성하는 역할만 수행하며, 각 모듈의 의존성 설치나 실행 환경 설정은 별도로 진행한다.

ave-client는 사용자 PC에서 실행되는 로컬 클라이언트로, 영상 및 분석 자료 수집, 스크립트 처리, 주요 구간 분석·추천, 사용자 인터페이스 및 최종 영상 렌더링을 담당한다. ave-server는 중앙 서버에서 사용자 인증, 외부 LLM 및 원격 STT 요청 중계, 일부 완료 작업 이력 관리를 수행한다.

ave-whisper-api는 RunPod 환경에서 WhisperX 기반 음성 전사와 단어 단위 시간 정렬을 수행하는 모듈이며, ave-dist는 Windows용 실행 파일과 포터블 배포본, 외부 도구 업데이터를 생성하는 역할을 담당한다.

이와 같이 프로젝트를 기능과 실행 환경에 따라 네 개의 저장소로 분리하여 로컬 영상 처리, 중앙 서버 기능, GPU 기반 음성 전사, 배포 환경을 독립적으로 개발하고 관리할 수 있도록 구성하였다.

3.1.3. 영상 및 분석 데이터 수집

본 시스템은 사용자가 입력한 YouTube URL을 기준으로 영상의 메타데이터와 처리 가

능 여부를 먼저 확인한 뒤, 분석에 필요한 원본 영상과 부가 자료를 사용자 PC에 수집하도록 구성하였다. 입력 영상은 일반 업로드 영상과 종료된 라이브 스트리밍 다시보기(VOD)를 대상으로 하며, 진행 중인 실시간 방송은 지원하지 않는다. 지원되지 않는 URL이나 영상 길이가 입력된 경우에는 분석 작업을 시작하지 않고 오류를 반환한다.

원본 영상은 yt-dlp를 이용하여 사용자 PC에 다운로드한다. 함께 수집되는 메타데이터에는 영상 제목, 설명, 채널, 업로드 날짜, 조회 수, 좋아요 수, 댓글 수, 태그, 카테고리, 자막 언어 목록 등이 포함되며, 영상 처리 조건 확인과 이후 분석 단계에서 활용된다.

영상 내용 분석을 위한 스크립트 자료로는 YouTube에서 제공되는 업로드 자막과 자동 생성 캡션을 사용할 수 있다. 사용 가능한 언어와 자막 자료가 존재하는 경우 사용자가 이를 선택할 수 있으며, 수집된 자막은 VTT 형식으로 저장한 뒤 각 문장의 시작 시각, 종료 시각 및 텍스트를 포함하는 세그먼트 형태로 변환한다. 자동 생성 캡션의 경우 반복적으로 나타나는 롤링 형태의 중복 문장을 정리하여 분석에 사용한다.

주요 구간 판단에는 시청자 반응 및 시청 행동과 관련된 부가 정보도 활용한다. 사용자가 해당 자료의 수집을 선택하고 YouTube에서 데이터가 제공되는 경우 댓글, 라이브 채팅 리플레이, 히트맵 및 챕터 정보를 수집한다. 댓글에서는 본문에 포함된 영상 타임스탬프를 추출하여 해당 시점의 관심도를 분석하는 데 활용하며, 라이브 채팅 리플레이는 시간에 따른 채팅 활성도의 변화를 계산하는 데 사용한다. 히트맵은 YouTube에서 제공하는 재시청 정도를 나타내는 시간대별 값을 이용하며, 챕터 정보는 영상의 내용 구조를 구분하는 자료로 활용한다.

이와 같이 시스템은 원본 영상뿐만 아니라 자막, 댓글, 채팅, 히트맵, 챕터 등 여러 종류의 자료를 함께 수집하도록 구성하였다. 다만 모든 영상에서 이러한 부가 정보가 제공되는 것은 아니므로, 실제로 확보된 자료만 이후 분석 단계에 사용한다. 이를 통해 특정 정보가 존재하지 않는 영상에서도 분석 과정을 계속 수행할 수 있도록 하였다.

3.1.4. 음성 전사 및 스크립트 처리

영상의 내용을 분석하기 위해서는 영상의 음성을 텍스트로 변환하고, 각 문장이 원본 영상의 어느 시점에 해당하는지를 확인할 수 있는 스크립트가 필요하다. 본 시스템에서는 사용자가 YouTube의 업로드 자막 또는 자동 생성 캡션을 선택한 경우 해당 자료를 스크립트로 사용하며, 자막을 사용할 수 없거나 사용자가 선택하지 않은 경우에는 WhisperX 기반 원격 음성 전사를 수행하도록 구성하였다. 자막과 자동 생성 캡션 사이에

는 고정된 우선순위를 두지 않고, 실제로 제공되는 자료 중 사용자가 원하는 언어와 종류를 선택할 수 있도록 하였다.

원격 음성 전사가 필요한 경우 먼저 yt-dlp를 이용하여 영상의 고품질 음원을 MP3 형태로 사용자 PC에 저장한다. 이후 FFmpeg를 이용하여 원격 전사용 음성 형식으로 변환한다. 전사 요청에 사용되는 음성 파일은 중앙 서버로 임시 업로드되며, 중앙 서버는 해당 파일을 제한된 시간 동안 HTTPS로 접근할 수 있도록 제공한다. RunPod의 음성 전사 Worker는 이 주소를 통해 음성 파일을 다운로드한 뒤 WhisperX를 실행한다.

현재 원격 전사 환경에서는 Whisper large-v3 모델을 사용하며, CUDA 기반 GPU와 float16 연산을 이용한다. 음성 전사 이후에는 전사 언어에 맞는 CTC 기반 정렬 모델을 적용하여 강제 정렬을 수행하고, 각 문장뿐만 아니라 개별 단어의 시작 및 종료 시각을 생성한다. 이를 통해 단순한 음성 인식 결과보다 실제 영상 편집에 활용하기 적합한 시간 정보를 확보하도록 하였다.

음성 전사 과정에서는 처리 시간을 조절할 수 있도록 1.0배, 1.5배, 2.0배의 전사 속도를 선택할 수 있다. 배속 전사를 사용하는 경우 RunPod Worker에서 FFmpeg를 이용하여 음성 재생 속도를 변경한 뒤 전사를 수행하며, 전사 결과의 타임스탬프를 다시 원본 영상의 시간축으로 환산한다. 따라서 배속을 적용하더라도 최종 스크립트의 시간 정보는 원본 영상의 위치와 대응하도록 구성하였다.

전사가 완료되면 중앙 서버를 통해 결과가 사용자 PC로 전달되고, 최종 분석용 스크립트는 각 문장의 시작 시각, 종료 시각, 텍스트를 포함하는 세그먼트 형태로 구성된다. WhisperX의 원본 전사 결과에는 단어별 타임스탬프도 포함되며, 이는 이후 문장 분할 과정에서 활용된다. 정상적으로 결과가 전달된 이후 중앙 서버에 임시 저장된 전사용 음성 파일과 작업 제어 정보는 삭제하도록 구성하였다.

WhisperX가 생성한 하나의 세그먼트가 지나치게 긴 경우에는 이후 LLM 분석과 자막 활용에 적합하도록 추가적인 분할 과정을 수행한다. 공백을 제외한 문자의 수가 30자 이상이고 두 개 이상의 단어를 포함하는 세그먼트를 분할 대상으로 하며, 먼저 인접한 단어 사이에 0.5초 이상의 무음 간격이 존재하는지를 확인한다. 이러한 명확한 시간적 경계가 있을 경우 해당 위치를 우선적으로 분할한다.

무음 기준으로 분할한 이후에도 세그먼트가 긴 경우에는 LLM을 이용하여 문장의 의미 및 문법 구조상 적절한 단어 경계를 판단한다. 분할된 세그먼트의 시작과 종료 시각은 WhisperX의 단어 단위 타임스탬프를 이용하여 원본 영상의 시간 흐름을 유지하도록 조

정한다. 다만 LLM이 적절한 분할 위치가 없다고 판단할 수 있으므로 모든 최종 세그먼트가 반드시 30자 미만으로 구성되는 것은 아니다.

이와 같은 과정을 통해 YouTube에서 제공되는 자막을 이용하는 경우와 WhisperX를 이용하여 직접 음성을 전사하는 경우를 모두 하나의 공통적인 스크립트 구조로 변환한다. 생성된 스크립트는 이후 LLM을 이용한 챗터 및 섹션 구성과 주요 구간 중요도 분석의 기본 입력 자료로 사용된다.

3.1.5. LLM 기반 내용 분석

본 시스템에서는 앞 단계에서 확보한 스크립트를 기반으로 영상의 내용 구조를 파악하고 각 구간의 내용적 중요도를 평가하기 위해 대규모 언어 모델(LLM)을 활용한다. LLM 요청은 사용자 PC에서 외부 API로 직접 전달하지 않고 중앙 서버를 통해 중계하며, 현재 시스템에서는 Gemini 3.5 Flash-Lite와 DeepSeek V4 Flash를 사용할 수 있도록 구성하였다. 두 모델의 분석 결과는 후속 프로그램에서 직접 처리할 수 있도록 구조화된 JSON 형식으로 반환받으며, 응답의 일관성을 높이기 위해 temperature는 0.1로 설정하였다.

장시간 영상의 스크립트는 입력되는 텍스트의 양이 크기 때문에 동일한 정보가 반복적으로 전달되거나 LLM이 원문을 다시 출력할 경우 불필요한 토큰 사용이 증가할 수 있다. 또한 자유 형식의 자연어 응답은 예상하지 못한 설명이나 형식이 포함되어 이후 자동 처리 과정에서 오류가 발생할 가능성이 있다. 이에 본 시스템에서는 각 분석 단계에서 필요한 최소한의 정보만 입·출력하도록 데이터 구조를 단순화하고, 원본 텍스트와 타임스탬프는 로컬에 유지한 상태에서 정수 ID를 통해 구간을 참조하도록 구성하였다.

먼저 분석 대상 스크립트의 각 문장에는 영상의 시간 순서에 따라 정수 ID를 부여한다. LLM에는 문장의 ID와 텍스트를 중심으로 전달하고, 해당 문장의 실제 시작 및 종료 시각은 로컬 시스템에서 별도로 관리한다. 실제 분석 입력 역시 시간 순서의 정수 id와 text를 중심으로 구성된다.

```
{  
  "id": integer,  
  "text": string  
}
```

LLM 스크립트 입력 구조

이와 같이 문장의 시간 정보를 매 요청마다 반복하여 전달하지 않고 ID를 이용해 참조함으로써 입력 구조를 단순화하였다. 이후 LLM이 반환한 ID는 로컬에 저장된 원본 스크

립트와 다시 연결하여 실제 영상의 시간 범위로 변환한다.

LLM 기반 내용 분석의 첫 단계는 전체 스크립트를 큰 주제 단위인 챕터로 구분하는 것이다. 자동 챕터 분할 방식을 사용하는 경우 전체 스크립트의 문장 ID와 텍스트를 LLM에 전달하고, 각 챕터의 시작 문장 ID와 종료 문장 ID, 내용 요약, 중요도 점수를 반환받는다. 자동 챕터 분석 결과는 start_id, end_id, summary, score로 구성된다.

```
[
  {
    "start_id": integer,
    "end_id": integer,
    "summary": string,
    "score": integer
  }
]
```

챕터 분석 출력 구조

start_id와 end_id는 해당 챕터에 포함되는 스크립트 문장의 범위를 의미하며, summary는 챕터의 주요 내용을 나타낸다. score는 해당 챕터의 내용적 중요도를 나타내는 0부터 1000 사이의 정수이며, 시스템에서는 이를 1000으로 나누어 0부터 1 사이의 chapter_llm_score로 변환한다. 이 값은 이후 다른 분석 정보와 함께 구간별 종합 점수를 계산하는 데 사용한다.

챕터 분석 결과에서는 원본 스크립트의 내용을 다시 출력하지 않고 시작·종료 ID만을 이용하여 문장의 범위를 나타내도록 하였다. 이를 통해 동일한 스크립트가 응답에 반복되는 것을 방지하고, 반환된 값이 실제 입력 범위 안에 존재하는지 비교적 단순한 방식으로 검증할 수 있다.

챕터 구성이 완료되면 각 챕터 내부의 스크립트를 다시 세부 의미 단위인 섹션으로 분할한다. 이 단계에서는 해당 챕터에 포함된 문장의 ID와 텍스트를 LLM에 전달하고, 의미적으로 하나의 구간을 이루는 시작 및 종료 문장 ID만 반환받는다.

```
[
  {
    "start_id": integer,
    "end_id": integer
  }
]
```

섹션 분할 출력 구조

섹션 분할 단계에서는 별도의 제목이나 요약, 원본 텍스트를 다시 출력하지 않고 start_id와 end_id만 반환하도록 구성하였다. 시스템은 반환된 ID 범위에 포함되는 원본

스크립트 문장을 로컬에서 다시 결합하여 섹션의 실제 내용을 구성하며, 첫 번째 문장의 시작 시각과 마지막 문장의 종료 시각을 해당 섹션의 실제 시간 범위로 사용한다. 또한 반환된 섹션들이 원본 챕터의 문장을 순서대로 누락이나 중복 없이 포함하는지 검증한다. 실제 구현에서도 섹션 자체의 별도 제목이나 요약은 생성하지 않는다.

섹션 분할이 완료되면 각 섹션의 내용적 중요도를 별도의 LLM 요청을 통해 평가한다. 이 과정에서는 해당 챕터의 요약 정보와 각 섹션의 ID 및 텍스트, 사용자가 선택한 판단 기준을 함께 전달한다. LLM은 각 섹션의 식별자와 중요도 점수만을 반환한다.

```
[
  {
    "id": integer,
    "score": integer
  }
]
```

섹션 중요도 출력 구조

id는 평가 대상 섹션을 식별하는 값이고, score는 해당 섹션의 내용적 중요도를 나타내는 0부터 1000 사이의 정수이다. 시스템에서는 반환된 점수를 1000으로 나누어 0부터 1 사이의 llm_score로 변환한다. 이 점수는 단독으로 최종 편집 구간을 결정하는 데 사용하지 않고, 이후 댓글, 채팅, 음량, 히트맵 등의 다른 분석 정보와 함께 종합 점수를 계산하는 데 활용한다.

중요도 평가에서도 섹션의 전체 텍스트나 판단 이유를 응답에 다시 포함하지 않고 id와 score만 반환하도록 하였다. 이와 같이 분석 결과를 후속 단계에서 실제로 필요한 값으로 제한함으로써 출력 토큰의 불필요한 증가를 줄이고, 자연어 설명이 구조화된 결과에 혼입되면서 발생할 수 있는 파싱 오류 가능성을 낮추도록 구성하였다.

LLM이 반환한 결과는 그대로 사용하지 않고 사전에 정의된 데이터 구조와 일치하는지 검증한다. JSON 형식으로 정상적으로 해석되는지 확인한 뒤, ID가 정수인지, 허용된 범위 안에 존재하는지, 챕터와 섹션에 누락이나 중복이 없는지 등의 조건을 검사한다. JSON 문법 오류 또는 해당 요청에 전달된 검증 조건을 만족하지 못하는 경우에는 직전 오류의 원인을 다음 요청에 포함하여 다시 생성을 요청하도록 구성하였다. 정상적으로 검증된 응답은 입력 데이터와 스키마를 기준으로 체크포인트에 저장하여 동일한 분석 요청이 반복되는 경우 기존 결과를 재사용할 수 있도록 하였다.

LLM은 챕터 및 섹션 구성과 중요도 평가뿐만 아니라 이후 주요 구간의 문맥을 보정하는 과정에서도 사용된다. 종합 점수가 높은 섹션이 주요 후보로 선정되면 해당 구간만으

로 내용을 이해하기 어려운지를 판단하기 위해 동일한 챕터 내부의 다른 섹션과의 관계를 추가로 분석하고, 내용 이해에 필요한 최소한의 연결 구간을 추천하도록 구성하였다.

이와 같이 LLM 기반 내용 분석 단계에서는 장시간 스크립트를 챕터와 섹션 단위로 구조화하고 각 구간의 내용적 중요도를 수치화하며, 이후 주요 구간 추천에 필요한 구조적 정보를 생성한다. 동시에 간결한 JSON 스키마와 응답 검증 과정을 적용하여 장시간 텍스트 처리 과정에서의 불필요한 데이터 교환을 줄이고, 후속 알고리즘에서 안정적으로 사용할 수 있는 형태의 분석 결과를 생성하도록 하였다.

3.1.6. 주요 구간 점수 산정 알고리즘

본 시스템에서는 영상의 주요 구간을 하나의 정보만으로 판단하지 않고, 앞 단계에서 생성한 각 섹션을 기준으로 내용적 중요도, 사용자 반응, 음향 변화 및 시청 행동 정보를 종합하여 점수를 계산한다. 평가에 사용하는 정보는 섹션 LLM 중요도, 챕터 LLM 중요도, 댓글 타임스탬프, 라이브 채팅 활성화도, 음량 변화, YouTube 히트맵의 총 여섯 종류이며, 각 점수는 최종적으로 0부터 1 사이의 값으로 변환하여 결합한다. 영상에 따라 제공되지 않는 정보가 존재할 수 있으므로 모든 신호가 반드시 사용되는 것은 아니며, 실제로 확보된 정보만 최종 점수 계산에 반영한다.

3.1.6.1. LLM 기반 내용 중요도

내용적 중요도는 섹션 단위 점수와 해당 섹션이 속한 챕터의 점수를 각각 사용한다. 섹션 중요도는 앞 절에서 설명한 것과 같이 챕터의 요약, 섹션별 텍스트 및 사용자가 선택한 판단 기준을 LLM에 전달하여 0부터 1000 사이의 정수로 평가한다. 반환된 값은 1000으로 나누어 0부터 1 사이의 `llm_score`로 변환한다. 챕터 중요도 역시 자동 챕터 분석 과정에서 0부터 1000 사이의 점수를 반환받고 같은 방식으로 `chapter_llm_score`로 정규화한다.

섹션과 챕터의 점수를 분리하여 사용하는 이유는 개별 구간 자체의 중요도와 해당 구간이 포함된 큰 주제의 중요도를 함께 반영하기 위함이다. 이에 따라 특정 섹션의 내용만 중요하게 평가된 경우뿐 아니라, 영상 전체에서 중요한 주제를 다루는 챕터에 포함된 섹션도 종합 점수에 영향을 받을 수 있도록 구성하였다.

3.1.6.2. 댓글 타임스탬프 점수

댓글 점수는 시청자가 댓글에 직접 남긴 영상 타임스탬프를 이용한다. 수집된 최상위 댓글에서 MM:SS 또는 HH:MM:SS 형태의 시간 정보를 탐색하고, 한 댓글 안에 동일한 시간이 반복되어 있는 경우 중복된 시각은 제거한다. 이후 타임스탬프가 포함된 댓글의 내용을 LLM으로 평가하여 해당 댓글이 나타내는 관심도를 0부터 1000 사이의 정수로 반환받고, 이를 1000으로 나누어 0부터 1 사이의 값으로 변환한다.

댓글에 포함된 타임스탬프가 특정 섹션의 시간 범위에 위치하면 해당 댓글의 관심 점수를 그 섹션에 반영한다. 하나의 섹션에 여러 댓글 타임스탬프가 포함되는 경우에는 그중 가장 높은 점수를 `comment_score`로 사용한다. 이는 동일한 구간에 여러 반응이 존재하더라도 댓글의 단순 개수만을 누적하기보다, 해당 시점에 대한 강한 관심 신호가 존재하는지를 반영하기 위한 방식이다.

3.1.6.3. 채팅 활성화 점수

라이브 스트리밍의 채팅량은 영상마다 시청자 수와 평상시 채팅 빈도가 크게 다를 수 있으므로 단순한 메시지 개수를 절대적인 기준으로 사용하지 않는다. 본 시스템에서는 영상 내부의 일반적인 채팅 수준과 비교하여 특정 시점의 채팅 활동이 얼마나 두드러지는지를 상대적으로 평가한다.

채팅 이벤트의 반응 시점과 영상 장면 사이의 차이를 고려하여 점수 계산 시 채팅 시간을 3초 앞당겨 사용한다. 또한 일부 사용자의 반복 메시지가 전체 활동도를 지나치게 높이는 것을 방지하기 위해 이동 구간 내 한 작성자의 기여를 최대 3건으로 제한한다. 해당 시점의 활동도는 제한된 메시지 수와 고유 작성자 수를 각각 50%씩 반영하여 계산한다.

$$A = 0.5M + 0.5U$$

여기서 M 은 작성자별 기여 제한을 적용한 메시지 수이고, U 는 해당 구간의 고유 작성자 수이다.

시스템은 5초 간격으로 채팅 활동을 표본화하고, 각 시점에서 직전 30초의 활동량을 그 이전 180초의 기준 구간과 비교한다. 최근 활동이 평상시 수준에 비해 얼마나 증가했는지를 나타내는 상대 비율과 영상 전체에서 해당 활동량이 어느 정도로 높은지를 함께 사용한다. 상대 증가 정도에 대한 점수를 60%, 최근 활동량의 영상 내 순위를 40% 반영하

고, 영상 내부의 분포를 기준으로 P50을 0, P95를 1로 대응시켜 정규화한다. 유효한 채팅 이벤트가 10개 미만인 영상에서는 채팅 점수를 생성하지 않는다.

각 섹션의 최종 채팅 점수는 해당 섹션 내부 표본 전체의 단순 평균만 사용하지 않고, 순간적으로 채팅이 크게 증가한 지점을 함께 반영하도록 구성하였다. 섹션 내 점수 중 상위 10%의 평균을 70%, 전체 표본의 평균을 30% 반영한다.

$$T = 0.7T_{top10\%} + 0.3T_{mean}$$

이를 통해 짧은 시간 동안 시청자 반응이 집중된 구간이 긴 섹션의 전체 평균에 의해 지나치게 희석되는 것을 줄이도록 하였다.

3.1.6.4. 음량 변화 점수

음량 역시 영상마다 기본적인 녹음 수준과 방송 환경이 다르므로 절대적인 음량 자체보다 평상시 대비 상대적인 증가를 중심으로 평가한다. 먼저 사용자 PC에 저장된 고품질 MP3 음원을 FFmpeg를 이용하여 처리하고, 1초 단위로 전체 채널의 RMS(Root Mean Square) 기반 dBFS 값을 계산한다. 순간적인 표본 변화의 영향을 줄이기 위해 연속된 3개 표본의 중앙값을 이용하여 값을 평활화한다.

각 시점의 음량은 직전 30초 구간의 중앙값과 비교한다. 실제 주요 음량 상승으로 판단하기 위해 현재 음량이 -45 dBFS 이상이고, 기준 음량보다 3 dB 이상 높으며, 이러한 상태가 2초 이상 연속되는 경우를 대상으로 한다. 이후 영상 전체의 음량 변화 분포에서 P50을 0, P95를 1로 대응시켜 상대 점수로 정규화한다.

섹션 점수로 변환하는 과정은 채팅 점수와 동일하게 상위 10% 표본의 평균을 70%, 전체 표본 평균을 30% 반영한다.

$$V = 0.7V_{top10\%} + 0.3V_{mean}$$

따라서 단순히 영상에서 가장 큰 순간 음량을 찾는 것이 아니라, 평소 수준보다 유의하게 음량이 증가한 시점과 그 변화가 섹션 내부에서 얼마나 두드러지는지를 함께 반영한다.

3.1.6.5. YouTube 히트맵 점수

YouTube에서 히트맵 정보가 제공되는 영상의 경우 시청자가 특정 구간을 반복적으로

시청한 정도를 주요 구간 판단에 활용한다. yt-dlp를 통해 수집되는 히트맵은 각 시간 구간의 시작 시각, 종료 시각 및 0부터 1 사이의 값을 포함한다. 시스템은 각 히트맵 구간의 중앙 시점을 표본 위치로 사용하고, 인접한 표본 사이의 값은 선형 보간하여 연속적인 시간축상의 값을 계산한다.

각 섹션에 대해서는 섹션의 시작 시점, 종료 시점 및 내부에 위치하는 히트맵 표본의 값을 확인하고, 그중 가장 높은 값을 heatmap_score로 사용한다. 따라서 섹션 전체 평균보다 해당 구간 내부에 반복 시청이 집중된 지점이 존재하는지를 우선적으로 반영한다. 히트맵 정보가 제공되지 않는 영상에서는 해당 점수를 생성하지 않는다.

3.1.6.6. 최종 종합 점수 계산

각 분석 정보에서 계산한 점수는 모두 0부터 1 사이의 범위로 맞춘 후 서로 다른 가중치를 적용하여 섹션별 최종 점수를 계산한다. 현재 시스템에서 사용하는 가중치는 표 1과 같다.

분석 정보	가중치
YouTube 히트맵	0.25
댓글 타임스탬프	0.20
섹션 LLM 중요도	0.18
챕터 LLM 중요도	0.15
채팅 활성화도	0.12
음량 변화	0.10

표 1 주요 구간 평가 정보별 가중치

모든 분석 정보가 존재하는 경우 섹션의 최종 점수 S 는 다음과 같이 계산한다.

$$S = 0.25H + 0.20C + 0.18L_s + 0.15L_c + 0.12T + 0.10V$$

여기서 H 는 히트맵 점수, C 는 댓글 점수, L_s 는 섹션 LLM 중요도, L_c 는 챕터 LLM 중요도, T 는 채팅 점수, V 는 음량 점수를 의미한다.

그러나 실제 YouTube 영상에서는 댓글, 라이브 채팅, 히트맵 등 일부 데이터가 제공되지 않는 경우가 존재한다. 본 시스템에서는 존재하지 않는 신호를 0점으로 처리하지 않는다. 만약 제공되지 않은 정보를 0으로 계산하면 해당 정보가 단순히 존재하지 않는 영상이 실제로 낮은 평가를 받은 것처럼 처리될 수 있기 때문이다. 따라서 사용할 수 있는 신호의 가중치만을 다시 합산하여 다음과 같이 정규화한다.

$$S = \frac{\sum_{i \in A} w_i \cdot \text{clip}(x_i, 0, 1)}{\sum_{i \in A} w_i}$$

여기서 A 는 해당 영상에서 사용할 수 있는 분석 정보의 집합, x_i 는 각 분석 점수, w_i 는 해당 점수에 설정된 가중치를 의미한다. 각 입력 점수는 0부터 1 범위로 제한한 뒤 계산에 사용하며, 최종 점수 역시 0부터 1 사이의 값을 갖는다. 어떤 분석 점수도 생성되지 않은 경우에는 해당 섹션의 최종 점수를 생성하지 않는다.

이와 같이 본 시스템은 스크립트의 내용적 중요도만을 기준으로 주요 장면을 선택하지 않고, 영상의 큰 주제에 대한 중요도, 시청자가 댓글을 통해 직접 지목한 시점, 실시간 채팅의 상대적 증가, 평상시 대비 음량 변화, 반복 시청 히트맵을 함께 반영한다.

3.1.7. 주요 구간 추천 및 문맥 보정

앞 절에서 각 섹션의 종합 점수 계산이 완료되면 시스템은 이를 바탕으로 최종 편집본에 포함할 주요 구간을 추천한다. 이 단계에서는 단순히 점수가 높은 섹션만을 선택하는 것이 아니라, 해당 구간을 이해하는 데 필요한 문맥을 함께 고려하도록 구성하였다. 따라서 종합 점수가 높은 섹션을 우선적인 기준 구간으로 선택하고, 같은 챕터 내부에서 내용 이해에 필요한 연결 섹션을 추가한 뒤 사용자가 설정한 목표 편집 길이를 기준으로 추천 결과를 구성한다.

각 섹션은 `final_score`가 높은 순서로 정렬하며, 점수가 같은 경우에는 원본 영상에서 먼저 등장하는 섹션을 우선한다. 이를 통해 여러 분석 정보에서 높은 평가를 받은 구간부터 편집 후보로 검토하도록 하였다.

정렬된 후보 중 현재 우선순위가 가장 높은 섹션을 기준 구간인 앵커(Anchor)로 선택한다. 그러나 해당 섹션만을 독립적으로 편집본에 포함할 경우 인물의 발언이나 사건의 원인, 앞선 설명 등이 누락되어 결과 영상의 흐름이 부자연스러울 수 있다. 이를 보완하기 위해 시스템은 앵커와 동일한 챕터에 속하는 다른 섹션들을 대상으로 문맥상 필요한 연결 구간이 존재하는지를 LLM을 통해 추가로 판단한다.

문맥 판단 요청에는 해당 챕터의 요약 정보와 기준이 되는 앵커 섹션의 위치, 동일 챕터 내부의 섹션 ID 및 텍스트를 전달한다. LLM에는 앵커 구간의 내용을 이해하기 위해 반드시 필요한 최소한의 연결 섹션만을 선택하도록 요청하며, 앞쪽 또는 뒤쪽에 위치한 섹션 모두 연결 후보가 될 수 있다. 반환된 섹션 ID는 실제 챕터에 존재하는 값인지 검증한 뒤 추천 결과에 포함한다.

```
{
  "required_section_ids": [
    integer
  ]
}
```

문맥 연결 섹션 출력 구조

이와 같이 문맥 보정 단계에서는 새로운 요약문이나 설명을 생성하는 대신 필요한 섹션의 ID만 반환하도록 구성하였다. 이는 앞선 LLM 분석 단계와 마찬가지로 출력 구조를 단순하게 유지하고, 반환 결과를 원본 섹션과 직접 연결할 수 있도록 하기 위한 것이다.

하나의 앵커에 대해 문맥상 필요한 섹션이 결정되면 앵커와 연결 섹션을 하나의 선택 집합에 추가한다. 이후 아직 선택되지 않은 후보 가운데 다음으로 점수가 높은 섹션을 새로운 앵커로 선택하고 같은 과정을 반복한다. 이미 선택된 섹션은 중복하여 추가하지 않으므로 하나의 연결 구간이 여러 앵커에 필요하다고 판단되더라도 최종 결과에는 한번만 포함된다.

추천 과정에서는 사용자가 설정한 목표 편집 길이를 선택 중단 기준으로 사용한다. 현재 시스템에서 목표 길이는 60초부터 최대 7,200초까지 설정할 수 있으며, 사용자 인터페이스에서는 원본 영상 길이와 7,200초 중 더 짧은 값을 최대값으로 제한한다. 기본 목표 길이는 원본 영상의 30%이다.

추천 알고리즘의 전체 흐름은 다음과 같다.

1. 스크립트를 문장 별 섹션으로 분리한다.
2. 각 섹션을 종합 점수 내림차순으로 정렬한다.
3. 가장 높은 점수의 섹션을 앵커로 선택한다.
4. 같은 챕터 내부에서 문맥상 필요한 연결 섹션을 LLM으로 판단한다.
5. 앵커와 연결 섹션을 추천 집합에 추가한다.
6. 선택된 구간의 전체 길이를 계산한다.
7. 목표 길이에 도달하거나 이를 초과하면 추천을 종료하며, 그렇지 않으면 다음 후보를 대상으로 과정을 반복한다.

현재 구현에서는 새 앵커와 연결 구간을 추가한 뒤 전체 선택 길이가 목표 길이를 초

과하는 경우, 이를 다시 잘라 목표 길이에 정확히 맞추지 않고 해당 상태에서 추천을 종료한다. 따라서 목표 길이는 정확한 출력 길이를 보장하는 값이 아니라 추천 구간을 어느 정도까지 선택할 것인지를 결정하는 기준으로 사용된다. 반대로 모든 후보를 선택하였음에도 충분한 구간이 존재하지 않는 경우에는 최종 추천 길이가 목표값보다 짧아질 수도 있다.

추천이 완료된 이후에는 선택된 섹션을 종합 점수 순서가 아니라 원본 영상의 시작 시각 순서로 다시 정렬한다. 이는 주요 구간의 중요도 순으로 영상을 재배치하지 않고, 원본 방송의 시간적 흐름을 유지하면서 불필요한 부분을 제거한 형태의 편집본을 생성하기 위함이다. 동일한 섹션 ID가 여러 번 선택된 경우에는 중복을 제거하여 한 번만 포함한다.

3.1.8. 사용자 편집 및 영상 렌더링

본 시스템에서는 AI가 추천한 주요 구간을 바로 최종 영상으로 확정하지 않고, 사용자가 추천 결과를 검토한 뒤 실제 편집본에 포함할 구간을 최종 선택하도록 구성하였다. 이는 동일한 영상이라도 사용자의 편집 목적에 따라 필요한 장면이 달라질 수 있고, 자동 분석만으로 모든 편집 의도를 정확하게 판단하기 어렵기 때문이다. 따라서 앞 절에서 생성된 추천 결과는 편집 초안으로 제공되며, 최종 렌더링은 사용자의 선택이 완료된 이후 수행한다.

사용자는 시스템이 추천한 섹션을 검토하여 최종 영상에 포함할 구간을 결정할 수 있다. 최종 선택이 전달되면 로컬 시스템은 전달받은 섹션 ID가 실제 분석 과정에서 생성된 후보에 존재하는지를 확인한 뒤, 유효한 구간만을 원본 영상의 시작 시각 순서로 정렬한다. 이를 통해 사용자 측에서 전달된 값만을 그대로 사용하는 것이 아니라 분석 결과에 존재하는 구간인지 다시 확인한 후 렌더링에 사용하도록 구성하였다.

선택된 구간 가운데 서로 겹치거나 끝과 시작이 맞닿아 있는 구간은 렌더링 전에 하나의 연속된 시간 범위로 병합한다. 예를 들어 하나의 선택 구간이 100초부터 120초이고 다음 구간이 120초부터 140초라면, 이를 별도의 두 영상으로 처리하지 않고 100초부터 140초까지의 하나의 범위로 처리한다. 이는 불필요한 구간 분리를 줄이고 최종 렌더링 과정의 구성을 단순화하기 위한 것이다.

영상 편집은 사용자 PC에서 FFmpeg를 이용하여 수행한다. 일반적인 클립 결합 방식처럼 선택된 구간마다 별도의 영상 파일을 생성한 뒤 다시 이어 붙이는 구조를 사용하지 않고, 원본 영상에 대해 한 번의 FFmpeg 처리 과정에서 선택된 시간 범위만 추출하도록

구성하였다. 따라서 여러 선택 구간을 하나의 필터 처리 과정에서 순서대로 연결하여 최종 영상을 생성한다.

선택된 영상 구간에는 앞 단계에서 확보한 스크립트를 이용하여 자막을 함께 생성한다. 원본 스크립트의 각 문장과 사용자가 선택한 영상 구간의 시간 범위를 비교하여 실제 결과 영상에 포함되는 문장만 추출하고, 선택되지 않은 구간이 제거된 만큼 자막의 시작 및 종료 시간을 새로운 영상 시간축에 맞게 다시 계산한다.

본 시스템을 통해 사용자는 일반 영상 편집 프로그램에서 각 구간을 직접 탐색하고 잘라 연결하는 과정을 최소화하면서도, 최종 편집 구간에 대한 결정권은 직접 유지할 수 있도록 하였다.

3.1.9. 서버 통신 및 데이터 관리

본 시스템은 대용량 영상의 다운로드, 분석 보조 처리 및 최종 렌더링은 사용자 PC에서 수행하고, 사용자 인증과 외부 AI 서비스 연동이 필요한 기능만 중앙 서버를 통해 처리하도록 구성하였다. 이에 따라 원본 영상과 완성 영상은 중앙 서버에 영구 저장하지 않고 사용자 PC에 보관하며, 중앙 서버는 인증, LLM 요청 중계, 원격 음성 전사 요청 관리 및 완료 작업 이력 저장을 담당한다.

사용자는 웹 UI에서 Google 계정을 이용하여 로그인하며, 중앙 서버는 Supabase를 통해 사용자 인증 정보를 확인한다. 이후 LLM 분석이 필요한 경우 사용자 PC는 Gemini 또는 DeepSeek API를 직접 호출하지 않고 중앙 서버에 요청을 전달한다. 중앙 서버는 외부 API 인증 정보를 관리하고 분석 요청을 중계한 뒤 결과를 다시 사용자 PC로 반환한다.

WhisperX 기반 음성 전사가 필요한 경우에는 사용자 PC에서 생성한 전사용 음성 파일을 중앙 서버에 임시 업로드한다. 중앙 서버는 RunPod가 해당 파일에 접근할 수 있도록 일시적으로 HTTPS 주소를 제공하고, 전사가 완료되면 결과를 사용자 PC에 반환한다. 전사에 사용된 임시 음성 파일은 작업 완료, 실패, 취소 또는 만료 시 삭제하도록 구성하였다.

작업 결과는 로컬과 중앙 서버에 서로 다른 범위로 저장한다. 사용자 PC의 SQLite에는 완료된 편집 작업의 기본 정보와 최종 영상 경로, 선택한 구간 등의 정보를 저장한다. 중앙 서버의 Supabase에는 사용자별 작업 이력과 일부 분석 및 추천 결과를 저장한다. 다만 전체 스크립트, 댓글·채팅 원문, 전체 히트맵, 원본 영상 및 최종 영상과 같은 대용량

또는 상세 자료는 완료 이력으로 중앙 서버에 영구 저장하지 않는다.

최종 영상이 정상적으로 생성되면 로컬 작업을 완료 상태로 저장하고, 이후 중앙 서버에 완료 이력을 동기화한다. 만약 영상 렌더링은 성공했지만 중앙 서버와의 동기화만 실패한 경우에는 이미 생성된 결과 영상을 유지하고 사용자에게 동기화 실패를 알리도록 구성하였다.

이와 같이 본 시스템은 대용량 영상과 상세 분석 자료는 사용자 PC를 중심으로 관리하고, 중앙 서버에는 인증과 외부 AI 서비스 연동, 작업 이력 관리에 필요한 정보만 전달하도록 구성하였다. 이를 통해 중앙 서버의 저장 및 네트워크 부담을 줄이면서도 여러 외부 서비스를 하나의 시스템에서 연동할 수 있도록 하였다.

3.1.10. 배포 구조

본 시스템은 역할별 운영을 위해 사용자 PC의 클라이언트, 중앙 서버, 원격 GPU 음성 전사 환경으로 구분하여 독립적으로 배포하였다.

사용자용 클라이언트는 Windows 환경을 대상으로 하며, PyInstaller를 이용하여 실행 파일 형태로 패키징한다. 배포본은 포터블 ZIP 형태로 제공할 수 있도록 구성하여 사용자가 별도의 Python 개발 환경을 설치하지 않고도 실행할 수 있도록 하였다.

영상 수집과 렌더링에 사용되는 yt-dlp, FFmpeg, FFprobe는 별도의 실행 파일로 관리하며, 업데이트를 통해 필요한 도구를 선택적으로 갱신할 수 있도록 하였다. 이를 통해 외부 도구의 변경에 대응하면서도 클라이언트 전체를 다시 배포해야 하는 부담을 줄였다.

중앙 서버는 Azure VM의 Ubuntu 환경에서 운영하며, FastAPI와 Caddy를 Docker Compose로 배포한다. 중앙 서버는 사용자 인증, LLM 및 원격 음성 전사 요청 중계, 작업 이력 관리를 담당하며, Caddy는 HTTPS 연결과 임시 음성 파일 제공에 사용한다.

WhisperX 기반 음성 전사는 RunPod의 서버리스 GPU 환경에 배포한다. 전사 요청이 발생할 때 필요한 GPU 자원을 사용하여 음성 전사와 단어 단위 시간 정렬을 수행하고, 결과를 중앙 서버를 통해 사용자 PC로 반환한다.

3.2. 시스템 동작 과정 및 사용자 인터페이스

본 절에서는 사용자가 실제로 시스템을 이용하는 순서에 따라 전체 작업 흐름과 사용

자 인터페이스를 설명한다. 본 시스템의 작업 과정은 로그인, 영상 입력 및 자료 수집, 스크립트와 분석 조건 설정, 주요 구간 추천, 사용자 선택 편집, 최종 렌더링의 순서로 진행된다. 각 단계에서는 이전 단계에서 수집하거나 생성한 정보를 확인한 뒤 다음 단계로 진행할 수 있도록 구성하였다.

3.2.1. LLM 공급자 선택 및 영상 URL 입력

사용자는 시스템 실행 후 Google 계정을 이용하여 로그인한다. 로그인 과정에서 사용자 인증이 완료되면 영상 분석 작업을 시작할 수 있는 화면으로 이동한다.

로그인이 완료되면 사용자는 분석에 사용할 LLM 공급자를 선택하고 분석 대상 YouTube 영상의 URL을 입력한다. 시스템은 입력된 URL을 확인하여 지원 가능한 형식의 영상인지 검증한 뒤 영상 정보 수집을 시작한다.

이 단계에서 선택한 LLM 공급자는 이후 챗터 생성, 섹션 분할, 중요도 평가 및 문맥 분석에 사용된다. 영상은 일반 업로드 영상과 종료된 라이브 스트리밍 다시보기를 대상으로 하며, 시스템의 입력 길이 조건을 만족하는 경우에 다음 단계로 진행한다.

The screenshot displays the 'Automatic Video Editor' web interface. At the top, there's a header with the title 'Automatic Video Editor' and a 'Google 로그인 필요' (Google Login Required) button. The main content area is a light blue box containing a white card titled '01 영상 URL 확인' (01 Check Video URL). Below the title, it says 'YouTube 영상 URL을 입력하고 사용할 LLM 엔진을 선택하세요.' (Enter the YouTube video URL and select the LLM engine to use). There are two input fields: 'LLM 엔진' (LLM Engine) with a dropdown menu showing 'DeepSeek-V4-Flash', and '영상 URL' (Video URL) with a text input field containing 'https://www.youtube.com/watch?v=...'. A red '정보 확인' (Check Info) button is to the right of the URL field. At the bottom of the interface, there's a progress bar labeled '대기 중' (Waiting) at 0%, and a red '다음' (Next) button.

그림 2 LLM 공급자 선택 및 YouTube URL 입력 화면

3.2.2. 영상 메타데이터 획득 및 추가 자료 선택

영상 URL이 확인되면 시스템은 먼저 기본 메타데이터를 수집한다. 여기에는 영상 개요, 설명, 챕터 정보 및 히트맵과 같이 분석에 활용할 수 있는 정보가 포함된다. 수집된 정보를 화면에 표시하여 사용자가 분석 대상 영상을 확인할 수 있도록 한다.

이후 사용자는 추가로 수집할 자료를 선택한다. 댓글, 라이브 채팅, 업로드 자막, 자동 생성 캡션 등이 해당하며, 실제 YouTube 영상에서 제공되는 자료만 선택할 수 있다. 모든 영상에 동일한 부가 자료가 존재하는 것은 아니므로 사용 가능한 항목은 영상에 따라 달라진다.

사용자가 필요한 항목을 선택한 뒤 다음 단계로 진행하면 해당 자료를 추가로 다운로드 하여 이후 분석 과정에 활용한다.

01 영상 URL 확인
YouTube 영상 URL을 입력하고 사용할 LLM 엔진을 선택하세요.

LLM 엔진: Gemini 3.5 Flash-Lite

영상 URL: https://www.youtube.com/watch?v=ksMRTIRp_Ss 정보 확인

개요 설명 챕터 히트맵

[이슈진단] 강대국의 최첨단 무기 개발 현 주소는? / 연합뉴스TV (Yonhapnews TV)

채널: 연합뉴스TV
조회수: 3,301,954
채널: 미지권
ID: ksMRTIRp_Ss

길이: 00:17:45
좋아요: 15,671
자막: 미지권
카테고리: People & Blogs

날짜: 2015-02-08
댓글: 336
업로드: 지원 ☒
대그: Yonhapnews TV

영상 정보를 확인했습니다. 추가 자료를 선택하거나 다음으로 진행하세요.

대기 중 0%

다음

그림 3 영상 메타데이터 확인 및 추가 자료 선택 화면

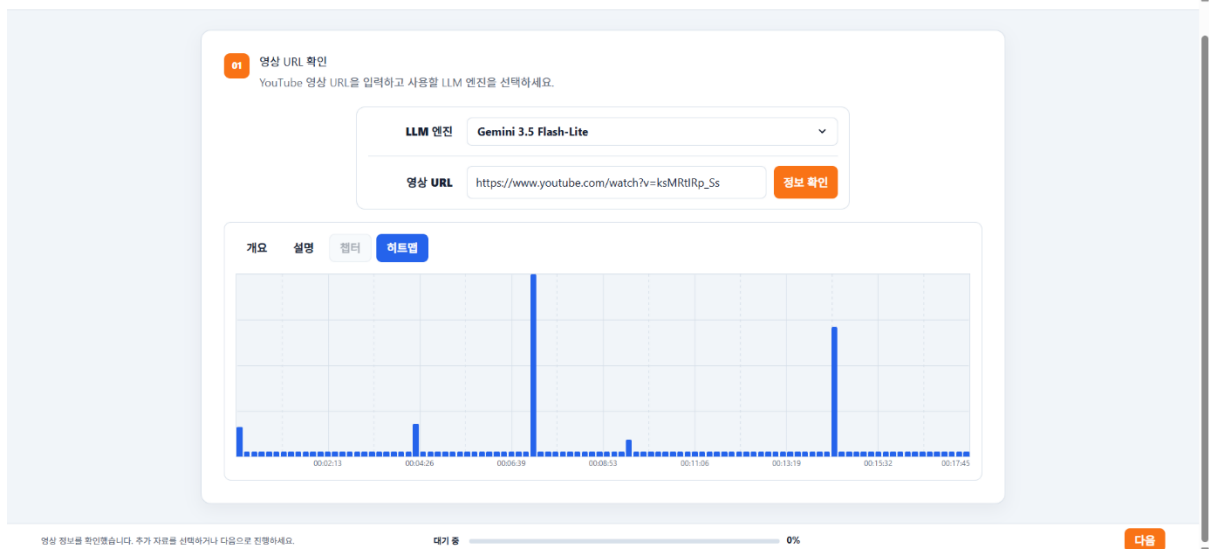


그림 4 영상 메타데이터(히트맵) 확인 화면

3.2.3. 댓글·채팅·자막 확인 및 Whisper STT 설정

추가 자료 수집이 완료되면 사용자는 다운로드한 댓글, 라이브 채팅, 업로드 자막 및 자동 생성 캡션의 내용을 확인할 수 있다. 이를 통해 실제 분석에 사용할 수 있는 자료가 정상적으로 확보되었는지 확인한다.

영상에 적절한 자막이나 캡션이 없거나 이를 사용하지 않는 경우에는 WhisperX 기반 STT를 사용할 수 있다. 이 단계에서 사용자는 음성 전사 사용 여부와 필요한 전사 설정을 지정한다. STT를 선택하면 시스템은 영상의 음성을 이용하여 스크립트를 생성하며, 기존 자막 또는 캡션을 사용하는 경우에는 별도의 전사를 수행하지 않는다.

따라서 사용자는 영상별 데이터 상태에 따라 기존 자막을 활용하거나 음성 전사를 수행하는 방식 중 적절한 스크립트 생성 방법을 선택할 수 있다.

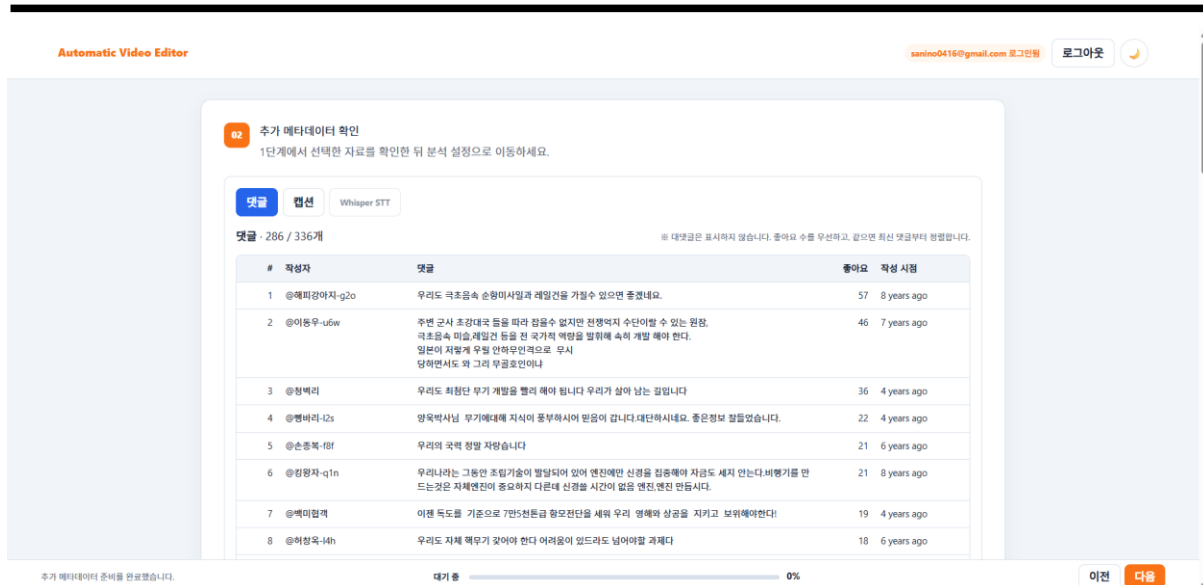


그림 5 수집 자료(댓글) 확인 화면

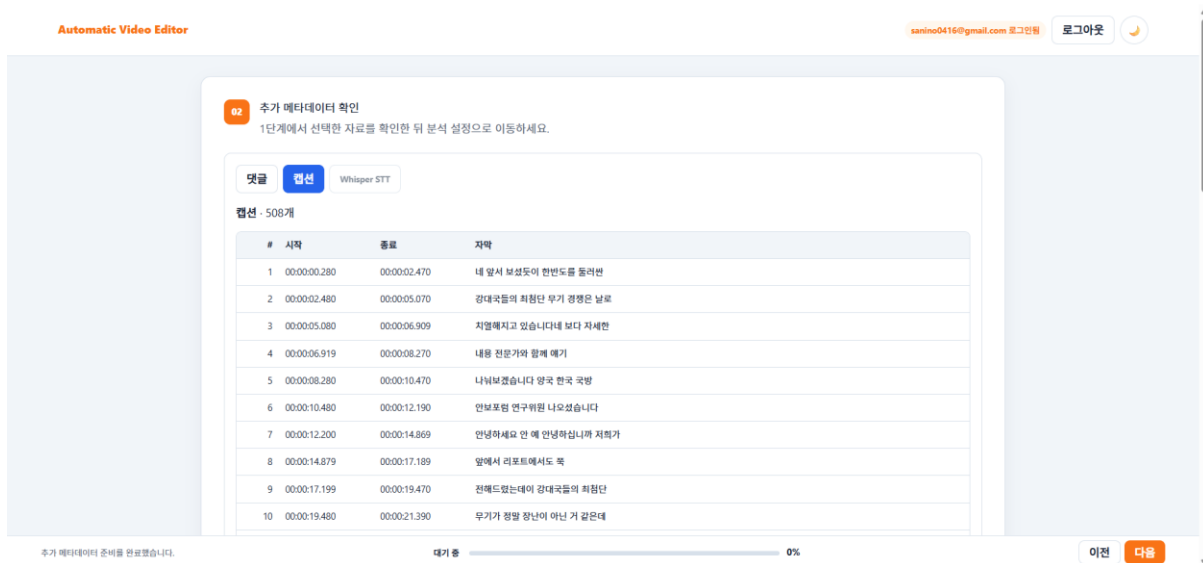


그림 6 수집 자료(캡션) 확인 화면

The image shows a web interface for Whisper STT settings. At the top, there are three tabs: '메뉴' (Menu), '캡션' (Caption), and 'Whisper STT' (selected). The main content area is titled 'Whisper STT 설정' (Whisper STT Settings) with a subtitle 'STT(Speech-to-Text)에 사용할 언어와 처리 옵션을 설정합니다.' (Set the language and processing options to be used for STT). The settings include:

- Whisper STT 사용**: A checkbox that is checked.
- 언어**: A dropdown menu set to '한국어' (Korean).
- 재생 배속**: A dropdown menu set to '1.0 (표준)' (1.0 (Standard)). A note below says '원본 음성을 빠르게 재생할수록 처리 시간은 줄지만 인식 정확도가 낮아질 수 있습니다.' (The faster you play back the original audio, the shorter the processing time, but the recognition accuracy may decrease).
- 초기 프롬프트**: A checkbox that is unchecked. A note below says '인식 결과의 문제, 문장 부호, 표현 방식을 유도할 때는 문맥을 입력합니다.' (When you want to guide the recognition results, punctuation, or expression style, enter the context).
- 찾워드**: A checkbox that is unchecked. A note below says '정확히 인식해야 할 고유명사를 원프로로 구분해 입력합니다.' (Separate proper nouns that must be recognized accurately with the original word as a prefix and enter them).

 At the bottom, there is a progress bar labeled '대기 중' (Waiting) with '0%' completion. On the left, it says '추가 메타데이터 준비를 완료했습니다.' (Additional metadata preparation is complete). On the right, there are buttons for '이전' (Previous) and '다음' (Next).

그림 7 Whisper STT 설정 화면

3.2.4. 스크립트 소스 및 분석 조건 설정

스크립트가 준비되면 사용자는 실제 분석에 사용할 스크립트 소스를 선택한다. 업로드 자막, 자동 생성 캡션 또는 WhisperX 전사 결과 중 사용 가능한 항목을 기준으로 분석 대상을 지정한다.

이와 함께 챕터 분할 방식을 설정하고, 주요 구간의 중요도를 판단할 때 사용할 점수 부여 기준을 프롬프트 형태로 설정할 수 있다. 이를 통해 동일한 영상이라도 사용자가 원하는 편집 목적에 따라 중요 구간의 판단 기준을 조정할 수 있다.

또한 스크립트는 시간 단위로 분리되어 표시되며, 사용자는 각 스크립트 구간의 내용을 확인하고 해당 시점의 원본 영상을 미리볼 수 있다. 이를 통해 실제 분석을 수행하기 전에 스크립트의 내용과 시간 정보가 영상과 적절하게 대응하는지 확인할 수 있다.



그림 8 스크립트 소스 및 분석 조건 설정 화면

3.2.5. 목표 길이 설정 및 챕터·점수 미리보기

내용 분석이 완료되면 생성된 챕터와 각 섹션의 분석 결과를 확인할 수 있다. 사용자는 챕터별 구간 구성과 섹션별 점수를 확인하여 시스템이 영상의 구조와 중요도를 어떻게 판단하였는지 검토할 수 있다.

이 단계에서 최종 편집 영상의 목표 길이를 설정한다. 목표 길이는 이후 AI가 주요 구간을 자동 선택할 때 사용하는 기준으로, 정확한 최종 영상 길이를 강제하는 값이 아니라 어느 정도의 분량까지 추천할 것인지를 결정하는 기준으로 사용된다.

사용자는 추천을 수행하기 전에 챕터 구성과 점수를 미리 확인할 수 있으므로 분석 결과가 편집 목적에 적합한지 검토한 뒤 자동 선택 단계로 진행할 수 있다.



그림 9 목표 편집 길이 설정 및 챕터-점수 미리보기 화면

3.2.6. AI 자동 선택 확인 및 선택 편집

설정된 목표 길이와 각 섹션의 점수를 기준으로 시스템은 주요 구간을 자동으로 선택한다. 높은 점수를 받은 섹션을 중심으로 추천하고, 내용 이해를 위해 필요한 경우에는 같은 챕터의 관련 구간을 함께 포함한다.

추천 결과는 최종 편집 결과로 확정되는 것이 아니라 사용자가 수정할 수 있는 초기 편집안으로 제공된다. 사용자는 AI가 선택한 각 구간을 확인하고 필요하지 않은 구간은 선택에서 제외할 수 있다. 반대로 자동 추천에 포함되지 않은 섹션도 직접 추가할 수 있으며, 챕터 또는 섹션 단위로 선택 상태를 변경할 수 있다.

화면에서는 현재 선택된 구간의 전체 길이를 확인할 수 있으며, 이를 참고하여 최종 편집 분량을 조정한다. 최종적으로 사용자가 렌더링에 포함할 구간을 확정함으로써 AI의 자동 추천과 사용자의 편집 판단을 함께 반영한다.

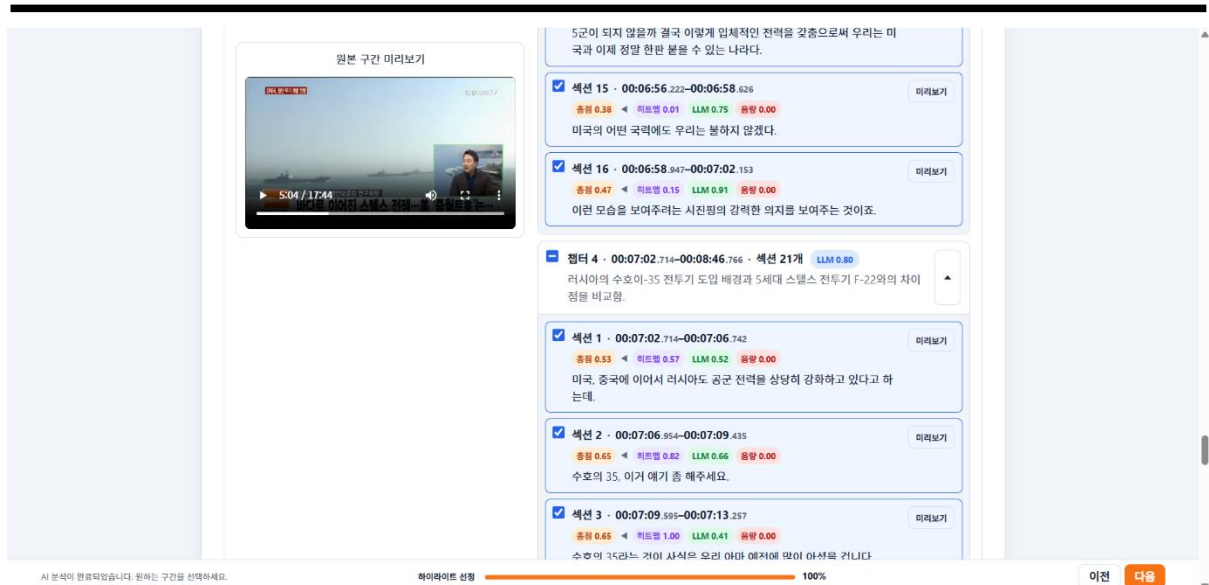


그림 10 AI 자동 선택 결과 및 편집 구간 수정 화면

3.2.7. 렌더링 및 결과물 확인

사용자가 최종 구간 선택을 완료하면 렌더링을 시작한다. 시스템은 선택된 구간을 원본 영상의 시간 순서에 따라 정리하고, FFmpeg를 이용하여 하나의 편집 영상으로 생성한다. 선택된 구간에 포함되는 스크립트는 새로운 영상 시간축에 맞게 변환되어 자막으로 합성된다.

렌더링 과정에서는 작업 진행 상태를 화면에서 확인할 수 있다. 렌더링이 완료되면 생성된 최종 영상을 재생하여 결과를 확인할 수 있으며, 편집 영상은 사용자 PC의 로컬 저장 공간에 보관된다.

사용자는 최종 영상에서 선택한 주요 구간이 의도한 순서로 연결되었는지, 자막이 정상적으로 표시되는지 등을 확인한다. 이를 통해 영상 입력부터 분석, AI 추천, 사용자 수정, 최종 영상 생성까지의 전체 작업이 하나의 사용자 인터페이스 흐름 안에서 완료된다.



그림 11 렌더링 진행 및 최종 결과물 확인 화면

4. 연구 결과 분석 및 평가

4.1. 목표 편집 길이에 따른 결과 분석

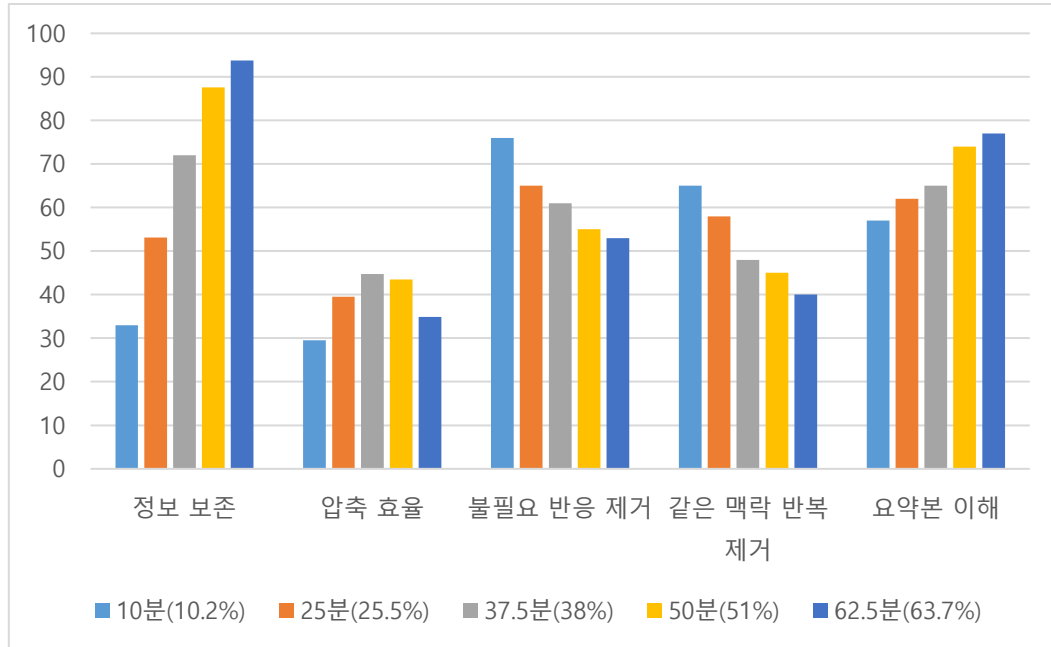


그림 12 목표 편집 길이에 따른 영상 요약 결과 비교

Input : 98분 길이의 정보 전달 영상(www.youtube.com/watch?v=UnWoCJPRjYg)

평가 지표 : GPT6-Astra에 표4(video_highlight_evaluation_prompt.md) 내용 입력

- 정보 보존: 원본의 핵심 정보가 중요도를 반영해 얼마나 보존되었는지
- 압축 효율: 압축률과 정보 보존율을 곱한 점수
- 불필요 반응 제거: 불필요한 추임새가 전달을 방해하지 않는 정도
- 같은 맥락 반복 제거: 불필요한 반복·중복 설명이 정리된 정도
- 요약본 이해: 원본을 보지 않고 요약본만으로 이해할 수 있는 정도

표 2 지표별 세부 설명

본 실험에서는 서로 다른 목표 시간으로 생성된 요약 영상의 특성을 비교하기 위해 정보 보존, 압축 효율, 불필요 반응 정리 수준, 같은 맥락의 반복 정리 수준, 요약본 이해도를 벤치마크 지표로 활용하였다.

평가 결과, 요약 시간이 짧을수록 불필요한 반응과 반복적인 내용은 효과적으로 제거되지만 핵심 정보와 문맥도 함께 손실되는 경향이 나타났다. 반대로 요약 시간이 길어질수록 정보 보존과 이해도는 높아졌으나, 원본에 포함된 반응과 반복 내용이 더 많이 남으면서 요약의 간결성과 압축 효율은 낮아졌다.

10분 요약본은 원본의 10% 정도로 가장 많이 축약한 형태이다. 불필요 반응 정리 점

수는 76점, 같은 맥락 반복 정리 점수는 65점으로 가장 높아 간결성이 우수하였다. 그러나 정보 보존은 33점, 압축 효율은 29.5점, 이해도는 57점으로 평가되었다. 이는 반복적이거나 불필요한 장면뿐만 아니라 내용을 이해하는 데 필요한 핵심 정보와 연결 문맥까지 상당 부분 제거되었음을 보여준다.

25분 요약본은 원본 대비 25.5%로, 10분 요약본보다 정보 보존이 53.1점으로 크게 상승하였으며, 압축 효율도 39.5점으로 개선되었다. 이해도 역시 62점으로 높아져 주요 내용을 파악할 수 있는 기본적인 문맥이 확보되었다. 동시에 불필요 반응 정리 점수 65점과 같은 맥락 반복 정리 점수 58점을 기록하여 비교적 간결한 구성도 유지하였다. 다만 핵심 설명이나 사례가 충분히 포함되지 않아 원본의 전체적인 논리와 세부 내용을 전달하기에는 일정한 한계가 있는 것으로 나타났다.

37.5분 요약본은 원본의 38%로, 정보 보존 72점, 압축 효율 44.7점, 이해도 65점을 기록하였다. 특히 압축 효율이 모든 시간 구간 가운데 가장 높았다는 점에서 영상 길이의 축소와 핵심 정보의 보존이 가장 효과적으로 결합된 구간으로 해석할 수 있다. 불필요 반응 정리 점수는 61점, 같은 맥락 반복 정리 점수는 48점으로 짧은 요약본보다 낮아졌지만, 핵심 내용과 이를 이해하는 데 필요한 설명 및 문맥은 비교적 안정적으로 유지되었다. 따라서 37.5분 요약본은 하이라이트 중심의 짧은 영상에서 내용 중심의 요약 영상으로 전환되는 중간적 특성을 보인다.

50분 요약본은 원본의 51%로, 정보 보존이 87.6점, 이해도가 74점으로 상승하여 원본의 주요 주장과 사례, 설명 과정을 대부분 전달하는 특성을 보였다. 그러나 압축 효율은 43.5점으로 37.5분 요약본보다 낮아졌으며, 불필요 반응과 같은 맥락 반복 정리 점수도 각각 55점과 45점으로 감소하였다. 이는 영상이 길어지면서 핵심 정보가 더 많이 보존된 동시에 요약 과정에서 제거되지 않은 반응이나 유사한 설명도 함께 증가했기 때문이다.

62.5분 요약본은 원본의 64%로, 정보 보존 93.8점과 이해도 77점으로 두 지표에서 가장 높은 점수를 기록하였다. 원본의 핵심 정보와 문맥이 대부분 유지되므로 내용 누락에 따른 이해 문제는 가장 적다. 반면 압축 효율은 34.9점으로 낮아졌고, 불필요 반응 정리 점수는 53점, 같은 맥락 반복 정리 점수는 40점으로 평가되었다. 즉, 정보 전달의 완전성은 가장 높지만 반복 설명과 부가적인 반응도 상대적으로 많이 남아 있어 요약 영상으로서의 간결성은 낮은 편이다.

각 시간대별 요약 영상의 특징을 정리하면 다음과 같다.

요약 시간	주요 특징
-------	-------

10분(10.2%)	강한 압축과 높은 간결성, 큰 정보 손실
25분(25.5%)	간결성을 유지하면서 기본적인 내용 전달
37.5분(38%)	정보 보존과 압축 효율이 균형을 이루는 구간
50분(51%)	높은 정보 보존과 이해도, 일부 반복 증가
62.5분(63.7%)	원본 내용을 대부분 보존하지만 압축 효과 감소

표 3 시간대별 요약 영상의 특징

종합하면 요약 시간의 증가는 단순히 영상 길이만 변화시키는 것이 아니라 요약본의 목적과 성격을 변화시키는 것으로 나타났다. 10분과 25분 요약본은 간결성과 빠른 시청에 중점을 둔 하이라이트형 결과물이며, 50분과 62.5분 요약본은 내용의 완전성과 이해도를 중시한 축약형 결과물이다. 37.5분 요약본은 이 두 유형의 경계에서 정보 보존과 시간 절약을 함께 확보한 균형형 결과물로 해석할 수 있다.

따라서 본 벤치마크 점수는 특정 요약 시간이 절대적으로 우수하다는 것을 판단하기 위한 기준이라기보다, 목표 시간에 따라 생성되는 요약 영상의 특성과 활용 목적을 구분하기 위한 비교 기준으로 활용할 수 있다.

영상 하이라이트 요약 품질 평가 프롬프트

당신은 영상 하이라이트·요약 품질을 정량적으로 평가하는 심사자입니다.

입력으로 **두 개의 SRT 또는 VTT 자막 파일**이 제공됩니다.

- 원본 영상 자막 파일
- 하이라이트·요약 영상 자막 파일

반드시 제공된 두 자막 파일을 직접 읽어 평가하세요. 자막 내부의 타임스탬프를 이용해 원본 영상과 요약 영상의 길이를 계산하세요.

평가 목표는 다음과 같습니다.

1. ****압축률****: 원본 대비 영상 길이가 얼마나 줄었는가
2. ****내용 보존율****: 원본의 핵심 정보가 요약본에 얼마나 보존되었는가
3. ****불필요한 추임새 정리 수준****: 전달 기능이 없는 추임새가 적절히 제거되었는가
4. ****반복·군더더기 정리 수준****: 불필요한 반복, 중복 설명, 무관한 잡담이 적절히 제거되었는가

5. ****흐름·독립적 이해 가능성****: 원본을 보지 않아도 요약본만으로 내용을 자연스럽게 이해할 수 있는가

평가 원칙

- 문장이나 단어의 단순 일치율이 아니라 ****의미와 핵심 정보의 보존 여부****를 평가하세요.
- 표현, 어순, 문장 구조가 달라도 같은 의미를 전달하면 보존된 것으로 판단하세요.
- 원본과 요약본의 타임스탬프는 서로 다른 시간축이므로 동일한 시각끼리 비교하지 마세요.
- 인사, 반복, 잡담, 말실수 등 핵심 내용과 관계없는 부분의 삭제는 정보 손실로 간주하지 마세요.
- 주요 사건, 주장, 결론, 중요한 사실·수치, 원인과 결과, 필수 맥락을 우선적으로 평가하세요.
- 원본에 없는 내용이 추가되거나 부정, 조건, 수치, 인과관계 또는 결론이 변경된 경우 해당 내용은 보존되지 않은 것으로 판단하세요.
- 자막에 존재하지 않는 영상 장면, 표정, 효과음, 화면 자료 또는 편집 효과를 추측하여 평가하지 마세요.
- 자막만으로 판단할 수 없는 요소는 임의로 추론하지 말고 평가 근거에서 제외하세요.
- 각 품질 점수는 다른 항목과 독립적으로 평가하세요. 예를 들어 내용이 많이 보존되었더라도 흐름이 부자연스러우면 흐름 점수는 낮게 부여할 수 있습니다.

평가 절차

1. 영상 길이 및 압축률

각 자막 파일의 최초 자막 시작 시각과 최종 자막 종료 시각을 기준으로 영상 길이를 초 단위로 계산하세요.

```
```text
```

```
duration = 최종 종료 시각 - 최초 시작 시각
```

```
length_ratio = summary_duration / original_duration
```

```
compression_rate = 1 - length_ratio
```

```
```
```

- 압축률은 0~100%로 표시하세요.
- 요약 영상이 원본보다 길어 압축률이 음수가 되는 경우, 실제 계산값을 그대로 표시하세요.
- 타임스탬프가 없거나 파싱할 수 없어 길이를 계산할 수 없는 경우 임의의 값을 만들지 마세요.

2. 핵심 정보 추출

원본 자막에서 영상 전체의 의미를 이해하는 데 필요한 ****핵심 정보 단위****를 식별하세요. 서로 다른 사실이나 주장을 지나치게 하나로 묶지 마세요.

각 핵심 정보 단위에 다음 기준으로 중요도를 부여하세요.

- 5: 핵심 메시지나 결론을 이해하는 데 필수
- 4: 매우 중요한 정보
- 3: 중요한 정보
- 2: 보조 정보
- 1: 생략 가능한 세부 정보

3. 내용 보존 평가

각 핵심 정보 단위가 요약 영상에 얼마나 보존되었는지 평가하세요.

- 1.0: 핵심 의미가 정확히 보존됨
- 0.75: 대부분 보존되었으나 일부 세부 정보가 빠짐
- 0.5: 핵심 의미의 일부만 보존됨
- 0.25: 간접적으로만 남아 의미 전달이 부족함
- 0.0: 누락되었거나 의미가 왜곡됨

다음 식으로 내용 보존율을 계산하세요.

```text

$$\text{content\_retention} = \sum(\text{importance} \times \text{retention}) / \sum(\text{importance})$$

$$\text{content\_retention\_score} = \text{content\_retention} \times 100$$

'''

결과는 0~100점으로 표시하세요.

#### ### 4. 압축 대비 정보 보존 평가

압축률과 내용 보존율을 함께 사용하여 다음 점수를 계산하세요.

```text

$\text{compression_efficiency_score} = \text{content_retention_score} \times \text{compression_rate}$

'''

이때 `compression\_rate`는 백분율이 아닌 0~1 비율을 사용하세요. 결과는 0~100점으로 표시하세요. 점수가 높을수록 **더 짧은 영상으로 원본의 핵심 정보를 많이 보존했다는 의미**입니다.

5. 불필요한 추임새 정리 평가

요약본에 남은 추임새가 정보 전달과 진행을 방해하는지 평가하세요.

- 전달 기능이 없는 습관성 표현, 의미 없는 감탄, 불필요한 맞장구 등이 반복되어 이해나 진행을 방해하면 감점하세요.
- 대화 관계, 화자의 태도, 감정, 분위기 또는 맥락을 이해하는 데 필요한 반응과 감정 표현은 보존되어도 감점하지 마세요.
- 단순히 추임새가 존재한다는 이유만으로 감점하지 말고, **불필요성과 방해 정도**를 기준으로 판단하세요.
- 원본에 추임새가 거의 없다면 요약본에도 문제가 없는 한 높은 점수를 부여하세요.

`filler\_cleanup\_score`를 0~100점으로 평가하세요.

- 90~100: 불필요한 추임새가 거의 없고 필요한 반응은 자연스럽게 보존됨
- 70~89: 일부 불필요한 추임새가 남지만 이해와 진행에 큰 지장이 없음
- 40~69: 불필요한 추임새가 여러 차례 남아 흐름을 방해함
- 0~39: 추임새가 과도하게 남아 요약·편집 효과가 매우 낮음

6. 반복·군더더기 정리 평가

요약본에서 같은 정보의 불필요한 반복, 중복 설명 및 무관한 잡담이 얼마나 정리되었는지 평가하세요.

- 동일하거나 사실상 같은 정보를 새 내용 없이 반복하면 감점하세요.
- 핵심 내용과 무관한 잡담이나 우회적인 설명이 요약본의 간결성을 해치면 감점하세요.
- 어려운 내용을 이해시키기 위한 재강조, 필요한 예시, 비교 또는 결론의 요약은 허용하며 감점하지 마세요.
- 반복의 존재 자체가 아니라 **새로운 전달 기능이 있는지**를 기준으로 판단하세요.

`redundancy\_cleanup\_score`를 0~100점으로 평가하세요.

- 90~100: 불필요한 반복과 군더더기가 거의 없이 간결함
- 70~89: 소수의 중복이나 잡담이 남지만 핵심 전달은 효율적임
- 40~69: 반복·군더더기가 자주 남아 압축 효과를 떨어뜨림
- 0~39: 중복과 무관한 내용이 과도하여 요약본으로서 비효율적임

7. 흐름·독립적 이해 가능성 평가

시청자가 원본을 보지 않고 요약본만 접한다고 가정하여 내용의 연결성과 완결성을 평가하세요.

다음 문제를 확인하세요.

- 필요한 배경이나 전제가 삭제되어 이해하기 어려움
- 대명사나 지시어의 대상이 불명확함
- 문장이 중간에서 잘리거나 앞뒤 발화가 부자연스럽게 연결됨
- 질문이 삭제되고 답변만 남아 발화 목적을 알 수 없음
- 주제나 사건이 설명 없이 갑자기 전환됨
- 결론이나 사건의 마무리가 없어 내용이 미완성으로 끝남

단, 짧더라도 필요한 맥락이 이미 요약본 안에 있거나 일반적인 상식만으로 명확히 이해할 수 있다면 감점하지 마세요.

`standalone\_coherence\_score`를 0~100점으로 평가하세요.

- 90~100: 원본 없이도 맥락, 연결, 결론을 자연스럽게 완전하게 이해할 수 있음
- 70~89: 일부 연결이 생략되었지만 전체 내용은 무리 없이 이해할 수 있음
- 40~69: 배경 부족이나 단절이 여러 차례 발생하여 일부 내용을 추론해야 함
- 0~39: 맥락과 연결이 심하게 훼손되어 요약본만으로 이해하기 어려움

출력 형식

반드시 아래 형식의 **유효한 JSON 객체만 반환**하세요. JSON 앞뒤에 설명, 제목, 코드 블록 표시 또는 주석을 추가하지 마세요.

```
```json
{
 "original_duration_seconds": 0.0,
 "summary_duration_seconds": 0.0,
 "compression_rate_percent": 0.0,
 "content_retention_score": 0.0,
 "compression_efficiency_score": 0.0,
 "filler_cleanup_score": 0.0,
 "redundancy_cleanup_score": 0.0,
 "standalone_coherence_score": 0.0,
 "key_information": [
 {
 "content": "핵심 정보",
 "importance": 5,
 "retention": 1.0,
 "assessment": "요약본에서 어떻게 보존 또는 누락되었는지 간단히 설명"
 }
],
 "editing_quality_assessment": {
 "filler_cleanup": "불필요한 추임새와 필요한 반응의 처리 상태를 간단히 설명",
 "redundancy_cleanup": "반복, 중복 설명, 잡담의 처리 상태를 간단히 설명",
 "standalone_coherence": "배경, 지시 대상, 문장 연결, 질의응답, 전환 및 결말의 완
```

결성을 간단히 설명"

```
},
 "missing_or_distorted_information": [
 "중요하지만 누락되거나 의미가 왜곡된 내용"
],
 "flow_issues": [
 "요약본의 독립적 이해를 방해하는 구체적인 문제"
],
 "summary": "압축률, 핵심 정보 보존 및 편집 품질에 대한 간결한 종합 평가"
}
...
```

#### ## 출력 규칙

- 모든 점수와 영상 길이는 소수점 첫째 자리까지 표시하세요.
- `importance`는 1~5의 정수, `retention`은 0.0, 0.25, 0.5, 0.75, 1.0 중 하나만 사용하세요.
- 문제가 발견되지 않은 배열은 빈 배열 `[]`로 반환하세요.
- JSON 문자열 내부의 줄바꿈이 필요하다면 올바르게 이스케이프하세요.
- 계산 결과와 개별 평가 근거가 서로 모순되지 않도록 검산하세요.
- 자막에 존재하지 않는 내용을 추측하거나 만들어내지 마세요.

표 4 video\_highlight\_evaluation\_prompt.md

## 4.2. 구성원별 역할 및 개발 수행 내역

- 김영빈 (조장)
  - React 기반 스크립트 분석 결과 확인, 편집 구간 결과 UI 구현
  - 사용자 정의 분석 기준의 추가·수정·삭제와 프롬프트 파일 관리 기능 구현
  - LLM 중요도·히트맵·댓글·채팅·음량 기반 점수 계산 구현
  - 원격 전사 옵션 설정·결과 처리 및 단어 타임스탬프 기반 자막 분할 기능 구현
  - 작업 진행 표시·취소·체크포인트 기반 분석 재개와 로컬 서버·트레이 실행

---

## 환경 구현

- 손재우
  - 스크립트 전처리와 LLM 분석용 입력 데이터 구성 기능 구현
  - 주제·문맥을 반영한 챗터·섹션 분할 및 중요도 평가 프롬프트·분석 흐름 구현
  - AI 분석 요청의 공동 처리와 응답 형식 검증, 동시 요청 조절 및 오류 재시도 기능 구현
  - 목표 길이에 따른 고득점 섹션 선정과 필수 문맥 연결을 반영한 편집 구간 자동 추천 구현
  - FFmpeg 기반 선택 구간 추출·결합·자막 합성 및 GPU·CPU 인코딩을 통한 결과 영상 생성 구현
- 염도엽
  - yt-dlp 기반 영상·음원·자막·댓글·채팅·히트맵·챗터 수집 및 자료 재사용 기능 구현
  - Supabase Auth의 Google 로그인과 사용자·작업 소유권 검증 및 데이터 접근 제어 구현
  - 중앙 API 서버와 Gemini·DeepSeek·RunPod 외부 서비스 호출·중계 및 자격 증명 관리 구현
  - WhisperX 전사·강제 정렬 GPU 서비스와 임시 오디오 호스팅·heartbeat·취소·만료 관리 구현
  - 로컬 파일·SQLite·Supabase 저장 경계와 완료 이력 동기화 및 중앙 서버·GPU worker 배포 환경 구성

### 4.3. 산학협력 멘토 의견 및 반영 결과

개발 과정에서 산학협력 멘토의 자문을 통해 시스템의 기술적 차별성, 추천 결과의 검증 방법, 사용자 편집 기능, 서비스 대상 및 저작권 문제에 대한 의견을 수렴하였다. 자문 내용 가운데 현재 개발 범위에서 적용 가능한 사항은 시스템 구조와 기능에 반영하였

---

으며, 추가적인 인프라와 데이터가 필요한 사항은 향후 연구 과제로 정리하였다.

#### 4.3.1. 독자적인 모델 구축 및 모델 고도화

멘토는 현재 시스템이 외부 STT 및 LLM 기술을 활용하는 구조이므로, 향후 정확도 향상과 기술적 차별화를 위해 자체 모델 구축이나 Fine-tuning을 검토할 필요가 있다는 의견을 제시하였다.

그러나 본 연구에서는 별도의 학습용 GPU 자원을 확보하지 못하여 자체 모델 학습이나 Fine-tuning까지 수행하기에는 현실적인 제약이 있었다. 이에 따라 현재 시스템은 Whisper 기반 음성 전사와 외부 LLM을 활용하는 구조를 유지하였다.

다만 본 시스템에서는 AI가 추천한 편집 구간을 사용자가 직접 검토하고 최종 선택하도록 구성되어 있다. 따라서 향후 사용자의 최종 편집 결과를 지속적으로 수집할 수 있다면, AI 추천 결과와 실제 사용자 선택 간의 차이를 학습 데이터로 활용할 여지가 있다. 충분한 데이터와 학습용 GPU 인프라가 확보될 경우 이러한 편집 이력을 기반으로 추천 모델을 학습하거나 기존 모델을 Fine-tuning하는 방향으로 발전시킬 수 있다.

따라서 독자적인 모델 구축은 현재 연구 범위에서는 수행하지 못하였으나, 사용자 편집 결과를 활용한 향후 데이터 확보 가능성을 확인하였다.

#### 4.3.2. 하이라이트 추출 결과 및 도메인별 검증

멘토는 시스템이 추천한 주요 구간의 타당성을 객관적으로 확인하기 위해 실제 시청자 반응이 높은 구간, 전문 편집자가 선택한 구간, 기존 자동 편집 서비스가 추출한 구간 등을 비교하는 정량적 검증이 필요하다고 제안하였다. 또한 게임, 토크, 요리 등 영상 유형에 따라 음성 비중과 텍스트 의존도가 달라질 수 있으므로 다양한 도메인을 대상으로 한 검증이 필요하다는 의견을 제시하였다.

본 연구에서는 개발 기간과 실험 여건을 고려하여 대규모 도메인별 데이터셋 구축이나 전문 편집 결과와의 구간 중복도 비교까지 수행하지는 못하였다. 대신 하나의 정보 전달 영상을 대상으로 서로 다른 목표 편집 길이의 결과물을 생성하고, 정보 보존, 압축 효율, 불필요 반응 제거, 같은 맥락 반복 제거 및 요약본 이해도를 기준으로 결과 특성을 비교하였다.

---

해당 실험을 통해 목표 편집 길이에 따라 정보 보존과 간결성 사이의 특성이 달라지는 것을 확인할 수 있었다. 향후에는 게임, 인터뷰, 강연, 토크 등 다양한 유형의 영상과 전문 편집 결과를 포함한 비교 데이터셋을 구축하여 추천 구간의 정확성과 특성을 추가로 검증할 필요가 있다.

#### 4.3.3. 사용자 수동 편집 기능

멘토는 LLM과 각 분석 신호를 통해 계산된 점수만으로 편집 구간을 자동 확정할 경우 대사가 잘리거나 앞뒤 맥락이 어색해질 수 있으므로, AI 추천 결과를 사용자가 직접 수정할 수 있는 수동 편집 기능이 필요하다고 제안하였다.

이를 고려하여 본 시스템에서는 AI의 추천 결과를 최종 편집본으로 바로 확정하지 않고, 사용자가 추천된 구간을 직접 확인한 뒤 최종 편집 구간을 결정하도록 구성하였다. 사용자는 추천된 섹션을 선택에서 제외하거나 추천되지 않은 섹션을 새롭게 추가할 수 있으며, 챕터 또는 섹션 단위로 선택 상태를 조정할 수 있다. 또한 현재 선택된 전체 영상 길이를 확인하면서 최종 편집 분량을 조정할 수 있다.

이와 함께 높은 점수의 구간만 독립적으로 선택하여 발생할 수 있는 문맥 단절을 줄이기 위해, 같은 챕터 내부에서 해당 구간을 이해하는 데 필요한 연결 섹션을 함께 추천하는 문맥 보정 과정도 적용하였다.

다만 현재 사용자 편집 기능은 섹션 단위의 선택 및 해제를 중심으로 제공하고 있으며, 타임라인에서 구간의 시작 시각과 종료 시각을 직접 미세 조정하는 기능은 구현하지 않았다. 해당 기능은 향후 보다 세밀한 편집이 필요한 사용자를 위한 개선 사항으로 고려할 수 있다.

#### 4.3.4. 서비스 대상 및 저작권 문제

멘토는 서비스의 주요 이용자가 자신의 영상을 편집하는 크리에이터인지, 타인의 영상을 재편집하는 제3자인지를 명확하게 설정할 필요가 있다고 제안하였다. 또한 제3자가 타인의 영상을 활용할 경우 발생할 수 있는 저작권 문제와 허용 범위에 대한 기준이 필요하다는 의견을 제시하였다.

본 연구에서는 시스템의 주요 사용 대상을 자신의 장시간 콘텐츠를 직접 편집하고자 하는 1인 크리에이터로 설정하였다. 따라서 타인의 콘텐츠를 재가공하기 위한 서비스보

---

다는 자신의 원본 영상을 효율적으로 편집하는 작업을 지원하는 도구를 목표로 하였다.

또한 원본 영상과 최종 결과 영상은 중앙 서버에 영구 저장하지 않고 사용자 PC에서 처리 및 보관하도록 구성하였다. 최종 렌더링 결과를 YouTube에 자동 업로드하지 않으며, 외부 게시 여부 역시 사용자가 직접 결정하도록 하였다.

#### 4.4. 종합 평가 및 한계

본 연구에서는 장시간 YouTube 영상 및 종료된 라이브 스트리밍 다시보기를 대상으로, 영상의 주요 내용을 자동으로 분석하고 사용자가 원하는 목표 길이를 기준으로 편집 후보 구간을 추천하는 시스템을 구현하였다. 영상 스크립트의 내용적 중요도뿐만 아니라 댓글 타임스탬프, 라이브 채팅, 히트맵, 음량 변화 등의 정보를 함께 활용하여 하나의 신호에만 의존하지 않도록 구성하였으며, 영상에서 제공되지 않는 정보는 최종 점수 계산에서 제외하도록 하였다.

또한 AI가 추천한 구간을 그대로 최종 결과로 확정하지 않고 사용자가 직접 검토하고 수정할 수 있도록 하였다. 사용자는 추천된 섹션을 제외하거나 추가할 수 있으며, 최종 선택된 구간을 기준으로 로컬 환경에서 렌더링을 수행한다. 이를 통해 자동 분석과 사용자 판단을 함께 활용하는 반자동 영상 편집 방식을 구현하였다.

목표 편집 길이에 따른 결과 비교에서는 영상 길이가 짧아질수록 불필요한 반응이나 반복 내용이 더 많이 제거되는 반면, 핵심 정보와 문맥도 함께 손실될 수 있음을 확인하였다. 반대로 편집 길이가 길어질수록 정보 보존과 이해도는 높아지지만 반복적인 설명이나 부가적인 내용이 더 많이 포함되어 압축 효과가 감소하였다. 따라서 하나의 고정된 편집 길이가 모든 목적에 적합하다기보다, 사용자가 원하는 결과물의 성격에 따라 목표 길이를 설정할 필요가 있음을 확인하였다.

다만 본 연구에는 몇 가지 한계가 존재한다. 첫째, 현재 주요 구간 판단은 외부 LLM과 사전 정의한 점수 계산 방식을 기반으로 하며, 사용자별 편집 성향이나 영상 유형에 따라 가중치를 자동으로 학습하는 기능은 제공하지 않는다. 또한 학습용 GPU 자원의 제약으로 자체 모델 학습이나 Fine-tuning을 수행하지 못하였다.

둘째, 댓글, 라이브 채팅, 히트맵과 같은 부가 자료는 모든 영상에서 제공되는 것이 아니므로 영상에 따라 활용 가능한 분석 신호가 달라진다. 해당 자료가 존재하지 않는 경우에도 분석은 가능하지만, 활용할 수 있는 판단 근거의 종류는 줄어들 수 있다.

---

셋째, 현재 사용자 편집 기능은 챕터 및 섹션 단위의 선택과 해제를 중심으로 제공된다. 따라서 사용자가 특정 대사의 시작과 종료 시점을 타임라인에서 직접 미세 조정하는 일반 영상 편집 프로그램 수준의 세부 편집 기능은 제공하지 않는다.

마지막으로 목표 편집 길이는 최종 결과의 정확한 길이를 보장하는 값이 아니라 주요 구간 추천을 위한 기준으로 사용된다. 섹션 단위로 구간을 선택하고 문맥상 필요한 연결 구간을 함께 포함하기 때문에 실제 추천 길이는 사용자가 설정한 목표보다 길거나 짧아질 수 있다.

이러한 한계에도 불구하고 본 연구에서는 장시간 영상의 수집, 스크립트 생성, 내용 분석, 다중 정보 기반 점수 계산, 문맥 보정, 사용자 최종 선택 및 렌더링까지의 전체 과정을 하나의 시스템으로 구현하였다. 이를 통해 장시간 영상을 처음부터 수동으로 검토해야 하는 작업을 줄이고, 사용자가 AI 추천 결과를 바탕으로 최종 편집 내용을 결정할 수 있는 영상 편집 지원 구조를 제시하였다.

---

## 5. 결론 및 향후 연구 방향

### 5.1. 결론

본 연구에서는 장시간 YouTube 영상 및 종료된 라이브 스트리밍 다시보기를 대상으로, 주요 내용을 분석하여 사용자가 설정한 목표 편집 길이를 기준으로 주요 구간을 추천하고 최종 영상으로 제작할 수 있는 영상 편집 시스템을 설계하고 구현하였다.

시스템은 영상의 내용을 단순히 음성 스크립트만으로 판단하지 않고, LLM 기반의 섹션 및 챗터 중요도와 함께 타임스탬프가 포함된 댓글, 라이브 채팅 반응, YouTube 히트맵, 음량 변화 등의 정보를 종합하여 각 구간의 중요도를 산정하도록 구성하였다. 영상에서 일부 정보가 제공되지 않는 경우에는 해당 신호를 0점으로 처리하지 않고, 사용 가능한 정보만을 기준으로 가중치를 다시 정규화하여 최종 점수를 계산하도록 하였다.

분석 과정에서는 자막 또는 자동 생성 캡션을 활용하거나, 필요한 경우 Whisper 기반 음성 전사를 통해 스크립트를 생성한다. 생성된 스크립트는 LLM을 이용하여 챗터와 섹션 단위로 분할하고 각 구간의 내용적 중요도를 평가한다. 이후 여러 분석 신호를 종합하여 높은 점수를 받은 구간을 주요 구간 후보로 선정하며, 해당 구간을 이해하기 위해 필요한 앞뒤 내용이 존재하는 경우 같은 챗터의 관련 섹션을 함께 추천하도록 하였다.

AI의 추천 결과를 최종 편집 결과로 바로 확정하지 않고 사용자가 직접 검토하고 수정하도록 구성한 것도 본 시스템의 주요 특징이다. 사용자는 추천된 구간을 제외하거나 추천되지 않은 구간을 추가할 수 있으며, 최종적으로 선택한 구간만을 원본 시간 순서에 따라 렌더링한다. 선택된 구간에 해당하는 자막도 새로운 영상 시간축에 맞게 변환하여 최종 영상에 합성하도록 구현하였다.

또한 대용량 원본 영상과 최종 결과 영상은 사용자 PC에서 처리하고 저장하는 로컬 중심 구조를 적용하였다. 중앙 서버는 사용자 인증, 외부 LLM 요청 중계, 원격 STT 처리 지원 및 일부 완료 이력 관리와 같은 역할을 담당하도록 하여 대용량 영상 파일을 중앙 서버에 지속적으로 저장하지 않도록 하였다.

목표 편집 길이에 따른 결과 분석에서는 편집 길이가 짧아질수록 불필요한 반응과 반복 내용이 더 많이 제거되는 대신 정보와 문맥의 손실 가능성이 커지고, 편집 길이가 길어질수록 정보 보존과 이해도가 높아지는 대신 압축 효과가 감소하는 경향을 확인하였다. 이를 통해 하나의 고정된 편집 길이를 일률적으로 적용하기보다 사용자가 원하는 결과물의 목적에 따라 목표 길이를 설정하는 방식이 적절함을 확인하였다.

---

결과적으로 본 연구에서는 영상 입력과 자료 수집부터 스크립트 생성, 내용 분석, 주요 구간 추천, 사용자 최종 선택, 자막 합성 및 영상 렌더링까지의 전체 과정을 하나의 작업 흐름으로 구현하였다. 이를 통해 장시간 영상을 직접 처음부터 검토하면서 주요 구간을 찾는 작업을 줄이고, AI 분석 결과를 참고하여 사용자가 최종 편집 내용을 결정할 수 있는 영상 편집 지원 방식을 제시하였다.

## 5.2. 향후 연구 방향

현재 시스템은 여러 종류의 정보를 종합하여 주요 구간을 추천하지만, 각 신호의 가중치는 사전에 설정된 값을 사용하며 영상 유형이나 사용자별 편집 성향에 따라 자동으로 변화하지 않는다. 향후에는 실제 사용자가 AI 추천 결과를 수정하고 최종적으로 선택한 편집 구간을 지속적으로 수집하여 추천 결과와 사용자 선택의 차이를 데이터로 축적할 수 있다. 충분한 학습 데이터와 인프라가 확보된다면 이를 기반으로 사용자 선택 경향을 학습하거나 기존 모델을 Fine-tuning하여 추천 방식의 개인화와 고도화를 검토할 수 있다.

사용자 편집 기능 역시 개선 가능성이 있다. 현재는 챕터와 섹션 단위로 추천 구간을 선택하거나 해제할 수 있으나, 사용자가 타임라인에서 각 구간의 시작 및 종료 시각을 직접 미세 조정하는 기능은 제공하지 않는다. 향후에는 AI가 제안한 구간을 기본 편집안으로 제공하면서 사용자가 프레임 또는 초 단위로 경계를 조정할 수 있는 기능을 추가하여 일반 영상 편집 도구에 가까운 편집 자유도를 제공할 수 있다.

분석 정보의 종류도 확장할 수 있다. 현재 시스템은 스크립트와 시청자 반응, 히트맵, 음량 등의 정보를 중심으로 분석하고 있으며 영상 프레임 자체에 대한 시각적 의미 분석은 수행하지 않는다. 향후에는 장면 전환, 인물 변화, 화면 내 텍스트, 객체 및 행동 변화와 같은 시각 정보를 추가로 분석하여 음성 정보가 적은 영상에서도 주요 장면을 판단할 수 있는 방향을 검토할 수 있다.

마지막으로 실제 서비스로 확장할 경우 저작권과 콘텐츠 이용 권한에 대한 정책을 보다 명확히 할 필요가 있다. 본 연구에서는 자신의 장시간 콘텐츠를 편집하려는 크리에이터를 주요 대상으로 설정하였지만, 실제 서비스에서는 타인의 영상 입력 가능성을 완전히 배제하기 어렵다. 따라서 이용 권한에 대한 안내, 서비스 이용약관, 결과물의 외부 배포 정책 등을 구체화하여 기술적 기능뿐만 아니라 실제 서비스 운영 측면에서도 보완할 필요가 있다.

---

## 6. 참고 문헌

- [1] 송정은. (2026, Feb. 16). “유튜버 1 인당 연 수입 7 천만, 4 년 만에 25%↑…상위 1%는 13 억,” 연합뉴스 [Online]. Available: <https://www.yna.co.kr/view/AKR20260215024600002>.
- [2] 강석영. (2023, Dec. 6). “[편집에 2~3 일 걸리는데] 10 분 영상 단가, 분당 고작 1 만원,” 매일노동뉴스 [Online]. Available: <https://www.labortoday.co.kr/news/articleView.html?idxno=218669>.
- [3] OpusClip. “How OpusClip Works,” OpusClip [Online]. Available: <https://www.opus.pro/how-does-opus-clip-work>.
- [4] Vizard. “Vizard.ai | AI Video Editing and Clipping Tool,” Vizard [Online]. Available: <https://vizard.ai/>.
- [5] Descript. “Generate Short Clips From Any Video with AI,” Descript [Online]. Available: <https://www.descript.com/clips>.